CCF-CSP 前三题备考讲义

第一题：基础实现　第二题：工具与算法　第三题：综合模拟

2026 年 10 月 11 日

参考代码按 C++17 编译，以普通数组、结构体、独立函数和 scanf/printf 为主。代码保留高亮与行号，支持复制正文；输入输出样例分别装框。

在线做题：曙梦 OJ；洛谷团队题单。

目录

第一部分　第一题：基础实现 **2**

1　程序基本功与做题技巧 **2**

2　例题速看 **5**

3　练习题 **21**

第二部分　第二题：工具与算法 **22**

1　基础算法 **22**

2　STL **33**

3　例题速看 **39**

4　练习题 **86**

第三部分　第三题：字符串解析与综合模拟 **87**

1　字符串解析与综合模拟基础 **87**

2　例题速看 **92**

3　练习题 **106**

第一部分　第一题：基础实现

1　程序基本功与做题技巧

1.1　程序基本功

程序基本功包括 C 语言的格式化输入输出、整型与浮点运算、函数的编写与调用、数组和结构体的使用。

1.1.1　格式化输入输出

scanf 把输入存入变量，普通变量前写地址符 &；printf 输出变量的值。格式符必须与类型匹配。

类型	示例声明	scanf	printf
int	`int n;`	`%d`	`%d`
long long	`long long s;`	`%lld`	`%lld`
double	`double x;`	`%lf`	`%.6f`（6 位小数）
char	`char c;`	`%c`	`%c`
字符数组	`char s[101];`	`%100s`	`%s`

```
int n; long long sum; double x; char c;
scanf("%d%lld%lf %c", &n, &sum, &x, &c);
printf("%d %lld %.6f %c\n", n, sum, x, c);
char word[101];
scanf("%100s", word);
```

`%c` 会读取空白，前面的空格表示先跳过空白。`%s` 读到空白就停止；上例最多读 100 个字符，另留一格存结束符。字符数组名表示首元素地址，不再加 &。

1.1.2　整型与浮点运算

整数除法向 0 截断，`5/2` 得到 2，`-5/2` 得到 -2；`%` 取整数余数。要保留小数，至少把一个操作数转为浮点数。

```
int a = 1000000000, b = 3;
long long product = 1LL * a * b;
double mean = (double)a / b;
int ceiling = a / b + (a % b != 0);
```

乘法先按操作数类型计算，再赋值；`1LL` 使乘法按 long long 进行。最后一行适用于 $a \geq 0, b > 0$，计算向上取整的商。double 能表示小数，但有舍入误差；输出精度由 printf 格式控制。

1.1.3 函数

函数用参数接收数据，用返回值给出结果。参数按值传递；要修改调用方的变量，可以传入它的地址。数组参数需要同时给出长度。

```
long long sum_array(const int a[], int n) {
    long long s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    return s;
}
void swap_int(int *x, int *y) {
    int t = *x; *x = *y; *y = t;
}
// Calls: sum_array(a, n); swap_int(&a[0], &a[1]);
```

const 表示函数不通过 a 修改数组。`*x` 访问地址 x 处的变量，所以交换结果会保留在原数组中。

1.1.4 数组

数组把同类型元素连续存放，`a[i]` 访问第 i 格。长度为 n 时，合法下标是 0 到 $n-1$；二维数组 `grid[i][j]` 分别用行、列下标访问。

```
const int N = 1005;
int a[N], grid[N][N]; // Global: initially zero.
int count[10] = {0};  // Local: explicitly initialized.
for (int i = 0; i < n; i++) scanf("%d", &a[i]);
```

全局数组自动清零，未初始化的局部数组不能直接读取。频率数组 `count[x]` 按数值 x 计数，要求 x 处于数组的合法下标范围。

1.1.5 结构体

结构体把同一对象的多个属性放在一起，通过点号访问成员；多个对象可以存入结构体数组。

```
struct Point { int x, y; char type; };
struct Point p[1005];
scanf("%d%d %c", &p[0].x, &p[0].y, &p[0].type);
int distance2 = p[0].x * p[0].x + p[0].y * p[0].y;
```

最后一行计算点到原点的距离平方，假定坐标绝对值不超过 1000，此时 int 足够。

1.2 做题技巧

1.2.1 数组容量与运算范围

数组容量由最大访问下标决定。若 $n \le 1000$ 且使用 1 下标，至少开 1001 格；若还访问 $n+1$，至少开 1002 格。字符串长度最多为 L 时，字符数组至少开 $L+1$ 格。

常用评测环境中 int 为 32 位，范围约为 $\pm 2.14 \times 10^9$；long long 为 64 位，范围约为 $\pm 9.22 \times 10^{18}$。把数据范围代入每一步运算：2×10^5 项、每项绝对值至多 10^9，和的绝对值至多 2×10^{14}，需要 long long；$10^{18} \times 100$ 则连 long long 也放不下。

1.2.2 时间复杂度与估算

时间复杂度表示操作次数随输入规模增长的数量级。先数循环执行次数，再忽略常数倍和低阶项；$O(n)$ 表示线性增长，$O(n^2)$ 表示平方增长。

一次遍历：$O(n)$。

```
long long sum = 0;
for (int i = 0; i < n; i++) sum += a[i];
```

执行 n 次加法。n 扩大 10 倍，操作次数也扩大 10 倍。

两次依次执行的遍历：$O(n)$。

```
long long sum = 0; int positive = 0;
for (int i = 0; i < n; i++) sum += a[i];
for (int i = 0; i < n; i++) if (a[i] > 0) positive++;
```

两个循环各执行 n 次，合计约 2n 次，忽略常数 2 后仍是 $O(n)$。

枚举所有数对：$O(n^2)$。

```
long long pairs = 0;
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
        if (a[i] == a[j]) pairs++;
```

比较次数是 $(n-1)+(n-2)+\cdots+1 = n(n-1)/2$。n 扩大 10 倍，次数约扩大 100 倍。

遍历 n 行 m 列：$O(nm)$。

```
long long sum = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++) sum += grid[i][j];
```

共 nm 次加法；若 m 始终是固定常数，便可记作 $O(n)$。

每次减半：$O(\log n)$。

```
int steps = 0;
for (int length = n; length > 1; length /= 2) steps++;
```

n=16 时依次处理 16、8、4、2，执行 4 次。通常约为 $\log_2 n$ 次，n 翻倍只多一次。

将最大规模代入估算：$n = 10^5$ 时，遍历约 10^5 次，数对枚举约 5×10^9 次，后者通常无法在几秒内完成。实际耗时还受操作类型、输入输出和机器速度影响。

1.2.3 空间复杂度与估算

空间复杂度表示存储量的增长数量级：n 项数组为 $O(n)$，$n \times m$ 矩阵为 $O(nm)$，固定数量的变量为 $O(1)$。深度 n 的递归还需要 $O(n)$ 调用栈空间。

实际内存约为元素个数乘以每个元素的字节数，再加上其他数组、容器和调用栈。1000×1000 的 int 矩阵约 3.8MiB，10^5 项 long long 数组约 0.76MiB。大数组通常放在全局，避免局部栈容量不足；结构体大小可用 sizeof 查询。

2 例题速看

2.1 打酱油

真题 · 第 11 次 CCF-CSP 认证，2017-09，第 1 题。打酱油（在线做题）。**题目与输入输出。**带 N 元买每瓶 10 元的酱油，可选择买 3 瓶赠 1 瓶或买 5 瓶赠 2 瓶，促销组之间不重复使用已赠送的瓶。求最多得到多少瓶。输入 N，输出最大瓶数。N 是 10 的整数倍且不超过 300。**读题思路。**先把钱换成可付费的瓶数，赠品不消耗钱，也不能把同一份购买重复参加两种促销。付费瓶数至多 30，枚举促销组完全够用。**做题思路与解答。**枚举买 5 赠 2 的组数 f。剩余付费瓶数为 r 时，尽可能组成买 3 赠 1 的组，剩下的按单瓶购买，得到 $7f + 4\lfloor r/3 \rfloor + r \bmod 3$。每种方案都有一个 f；固定 f 时多组成三瓶组只会增加赠品，所以逐项取最大值必然包含最优方案。也可证明优先五瓶组的贪心，但本题的小规模枚举不需要额外假设。

输入输出样例

样例输入

```
40
```

样例输出

```
5
```

输入输出样例

样例输入

```
80
```

样例输出

```
11
```

复杂度与边界。时间 $O(N/50 + 1)$，额外空间 $O(1)$。0 元时答案 0；余下 4 个付费瓶应组成三瓶组加单瓶。

完整参考实现 49_soy.cpp

```cpp
#include <cstdio>
int main() {
    int money; scanf("%d", &money);
    int paid = money/10, answer = 0;
    for (int five = 0; five*5 <= paid; five++) {
        int rest = paid-five*5;
        int bottles = five*7+(rest/3)*4+rest%3;
        if (bottles > answer) answer = bottles;
    }
    printf("%d\n", answer);
    return 0;
}
```

2.2 跳一跳

真题 · 第 13 次 CCF-CSP 认证，2018-03，第 1 题。跳一跳（在线做题）。**题目与输入输出。**输入若干个 0、1、2，数量不超过 30；0 仅在末尾出现，表示游戏结束。1 表示跳到方块非中心，得 1 分；2 表示跳到中心，若是第一次跳或上次得 1 分，则得 2 分，否则比上次多 2 分。输出总得分。**读题思路。**输入没有 n，由 0 终止；连续中心得分为 2、4、6 等，遇到 1 后必须重置。末尾 0 不计分。**做题思路与解答。**last 表示上一跳实际得分，初值 0。读到 1 令 last=1；读到 2 且 last 不超过 1 时令 last=2，否则加 2。然后累加 last。每次更新恰好实现规则，因此处理完前缀后总分始终正确。

输入输出样例

样例输入

```
1 1 2 2 2 1 1 2 2 0
```

样例输出

```
22
```

复杂度与边界。时间与输入数量成正比，空间 $O(1)$。输入首项为 0 时总分为 0；不要把 last 当作上一次输入的 1 或 2。

完整参考实现 50_jump.cpp

```cpp
#include <cstdio>
int main() {
    int x, last = 0, answer = 0;
    while (scanf("%d", &x) == 1 && x != 0) {
        if (x == 1) last = 1;
        else if (last <= 1) last = 2;
        else last += 2;
        answer += last;
    }
    printf("%d\n", answer);
    return 0;
}
```

2.3 线性分类器

真题 · 第 19 次 CCF-CSP 认证，2020-06，第 1 题。线性分类器（CSP202006A）。**题目与输入输出。**给出 n 个点，每点含整数坐标 x、y 以及类别 A 或 B，两类都存在。随后给 m 条直线 $\theta_0+\theta_1 x+\theta_2 y=0$；保证没有给定点恰在直线上。若同类点全部位于同一侧、异类点位于另一侧，输

出 Yes，否则 No，每条直线一行。输入首行 n、m，随后 n 行点、m 行三个系数。$1 \le n \le 1000$，$1 \le m \le 20$，坐标和系数绝对值不超过 10^4。**评测限制**。曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路**。必须把一条线与所有点比较；不是检查两类各找一个代表就结束。Yes 和 No 的大小写要精确。两侧由表达式符号区分，无需求斜率，因而竖直线也适用。**做题思路与解答**。用第一个点确定“正侧应该是哪一类”，再检查每个点的符号是否与类别相符。如果有反例，这条线不满足。代入式使用 long long 计算；原范围下 int 也能容纳，但显式提升类型便于保持安全的乘法习惯。正确性来自直线两侧的表达式符号相反，以及逐点检查完整覆盖了分类条件。

输入输出样例

样例输入

```
9 3
1 1 A
1 0 A
1 -1 A
2 2 B
2 3 B
0 1 A
3 1 B
1 3 B
2 0 A
0 2 -3
-3 0 2
-3 1 1
```

样例输出

```
No
No
Yes
```

复杂度与边界。时间 $O(nm)$，空间 $O(n)$。结构体记录三项属性；读类别用格式串中的空格跳过之前的换行。

完整参考实现 38_classifier.cpp

```cpp
#include <cstdio>
struct Point { int x, y; char type; } p[1005];
int main() {
    int n, m; scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++)
        scanf("%d%d %c", &p[i].x, &p[i].y, &p[i].type);
    while (m--) {
        int c, a, b; scanf("%d%d%d", &c, &a, &b);
        long long first = c + 1LL*a*p[0].x + 1LL*b*p[0].y;
        char positive = first > 0 ? p[0].type :
                        (p[0].type == 'A' ? 'B' : 'A');
        bool ok = true;
        for (int i = 0; i < n; i++) {
            long long v = c + 1LL*a*p[i].x + 1LL*b*p[i].y;
            if (v == 0 || ((v > 0) != (p[i].type == positive)))
                ok = false;
        }
        puts(ok ? "Yes" : "No");
    }
    return 0;
}
```

2.4 检测点查询

真题 · 第 20 次 CCF-CSP 认证，2020-09，第 1 题。检测点查询（在线做题）。**题目与输入输出。**输入 n、居民坐标 X、Y，随后 n 行检测点坐标，编号按输入顺序从 1 开始。输出最近的三个点编号，每个一行；同距离时编号小者在前。$3 \le n \le 200$，坐标绝对值不超过 1000。**读题思路。**要按距离、编号两层比较。无需真的开方，平方距离与距离具有相同顺序。只取三个点，三轮扫描就够用。**做题思路与解答。**计算各点平方距离。每轮从未选点中找最小距离者，按编号升序扫描且只在距离严格减小时替换，就自动保留同距离最小编号。输出并标记已选，重复三轮。归纳可知每轮得到剩余点中的第一名，三轮恰好得到所需前三名。

输入输出样例

样例输入

```
3 2 2
2 2
2 3
2 4
```

样例输出

```
1
2
3
```

输入输出样例

样例输入

```
5 0 1
-1 0
0 0
1 0
0 2
-1 2
```

样例输出

```
2
4
1
```

复杂度与边界。时间 $O(3n) = O(n)$，数组空间 $O(n)$。平方距离最大为 8×10^6，此范围 int 可装下；代码用 long long 并先提升差值类型。

完整参考实现 `51_nearest.cpp`

```cpp
#include <cstdio>
long long dist[205];
bool used[205];
int main() {
    int n, X, Y; scanf("%d%d%d", &n, &X, &Y);
    for (int i = 1; i <= n; i++) {
        int x, y; scanf("%d%d", &x, &y);
        long long dx = 1LL*x-X, dy = 1LL*y-Y;
        dist[i] = dx*dx+dy*dy;
    }
    for (int round = 0; round < 3; round++) {
        int best = 0;
        for (int i = 1; i <= n; i++)
            if (!used[i] && (best == 0 || dist[i] < dist[best])) best = i;
        printf("%d\n", best); used[best] = true;
    }
    return 0;
}
```

2.5 田地丈量

真题 · 第 29 次 CCF-CSP 认证，2023-03，第 1 题。田地丈量（CSP202303A）。**题目与输入输出**。输入 n、a、b，随后 n 行现有田地矩形的左下与右上坐标。现有田地互不重叠（交集面积为 0），目标区域为 $[0, a] \times [0, b]$。求目标区域内已有田地面积。$n \le 100$，所有坐标绝对值不超过 10^4，a、b 为正。输出面积。**读题思路**。统计连续面积，边接触贡献 0，不能加 1。现有田地互不重叠，所以各块与目标的交面积可以直接相加；若取消这一保证，则不能简单累加。**做题思路与解答**。每块田地裁剪到目标：横向左端取 $\max(x_1, 0)$、右端取 $\min(x_2, a)$，纵向同理。有正宽、正高才累加其乘积。两个一维交区间的笛卡尔积正好是矩形交集；原田地互不重叠，因此求和无重复覆盖。

输入输出样例

样例输入

```
4 10 10
0 0 5 5
5 -2 15 3
8 8 15 15
-2 10 3 15
```

样例输出

```
44
```

复杂度与边界。时间 $O(n)$，额外空间 $O(1)$，可边读边算。面积与累加值用 long long，乘法前提升类型。

完整参考实现 52_land.cpp

```cpp
#include <cstdio>
int main() {
    int n, a, b; scanf("%d%d%d", &n, &a, &b);
    long long answer = 0;
    while (n--) {
        int x1, y1, x2, y2; scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
        int left = x1 > 0 ? x1 : 0, down = y1 > 0 ? y1 : 0;
        int right = x2 < a ? x2 : a, up = y2 < b ? y2 : b;
        if (left < right && down < up) answer += 1LL*(right-left)*(up-down);
    }
    printf("%lld\n", answer);
    return 0;
}
```

2.6 小中大

真题 · 第 16 次 CCF-CSP 认证，2019-03，第 1 题。小中大（在线做题）。**题目与输入输出**。输入 $n \le 10^5$ 以及 n 个已按升序或降序排列的整数，可重复，绝对值不超过 10^7。输出最大值、中位数、最小值，空格分隔；整数不带小数，非整数保留 1 位小数。**读题思路**。已经有序，无需重排；升序和降序都能由首尾取得极值。奇数长度中位数是一项，偶数长度是中间两项均值。**做题思路与解答**。首尾取 min、max，中间位置按奇偶取值。先计算中位数的两倍 twice；偶数时直接输出 twice/2，否则输出 twice/2.0 保留一位。这使负半整数也正确，避免整数除法丢掉 0.5。

输入输出样例

样例输入

```
3
-1 2 4
```

样例输出

```
4 2 -1
```

输入输出样例

样例输入

```
4
-2 -1 3 4
```

样例输出

```
4 1 -2
```

复杂度与边界。读入时间 $O(n)$、数组空间 $O(n)$。可以只保留首尾及中间项把额外空间降为 $O(1)$。n=1 时三个输出值相同。

完整参考实现 53_small_middle_large.cpp

```cpp
#include <cstdio>
int a[100005];
int main() {
    int n; scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    int low = a[0] < a[n-1] ? a[0] : a[n-1];
    int high = a[0] > a[n-1] ? a[0] : a[n-1];
    long long twice = n%2 ? 2LL*a[n/2] : 1LL*a[n/2-1]+a[n/2];
    printf("%d ", high);
    if (twice % 2 == 0) printf("%lld", twice/2);
    else printf("%.1f", twice/2.0);
    printf(" %d\n", low);
    return 0;
}
```

2.7 数组推导

真题 · 第 23 次 CCF-CSP 认证，2021-09，第 1 题。数组推导（在线做题）。**题目与输入输出**。非负整数数组 A 的前缀最大值为 $B_i = \max(A_1, \ldots, A_i)$。已知单调不降的 B，求 A 的元素和最大值及最小值，各输出一行，最大值在前。输入 n、n 项 B；$1 \le n \le 100$，$0 \le B_i \le 10^5$。**读题思路**。B 是前缀最大值，不是前缀和。B 上升时当前 A 被迫等于新值；B 不变时 A 可小到 0。**做题思路与解答**。最大和为 $\sum B_i$，因为各项都可选为 B 对应值，且不能超过该值。最小和只保留第一项及严格上升处的 B 值，其他 A 取 0。上升时此前元素不可能提供新最大值，必须由当前 A 提供；这些强制项已足以实现全部 B，所以该和最小。

输入输出样例

样例输入

```
6
0 0 5 5 10 10
```

样例输出

```
30
15
```

输入输出样例

样例输入

```
7
10 20 30 40 50 60 75
```

样例输出

```
285
285
```

复杂度与边界。时间 $O(n)$，额外空间 $O(1)$。B 首项可为 0，全 0 时两个答案都是 0。

完整参考实现 `54_derive.cpp`

```cpp
#include <cstdio>
int main() {
    int n, previous = 0; scanf("%d", &n);
    long long largest = 0, smallest = 0;
    for (int i = 0; i < n; i++) {
        int b; scanf("%d", &b);
        largest += b;
        if (i == 0 || b > previous) smallest += b;
        previous = b;
    }
    printf("%lld\n%lld\n", largest, smallest);
    return 0;
}
```

2.8 如此编码

真题 · 第 27 次 CCF-CSP 认证，2022-09，第 1 题。如此编码（在线做题）。**题目与输入输出**。每题有 $a_i \geq 2$ 个选项，正确选项 $0 \leq b_i < a_i$。设 $c_0 = 1, c_i = \prod_{j=1}^{i} a_j$，编码 $m = \sum_{i=1}^{n} c_{i-1} b_i$。输入 n、m 及 n 项 a，恢复并输出 n 项 b，空格分隔。$n \leq 20, c_n \leq 10^9, 0 \leq m < c_n$。**读题思路**。这是不同位使用不同进制的编码。题面给出长公式，但可以从最低位逐次取余，避免反复计算前缀积。**做题思路与解答**。对 a_1 取余，后面所有项都是 a_1 的倍数，所以 $b_1 = m \bmod a_1$。再令 $m = \lfloor m/a_1 \rfloor$，得到余下位的同类编码。依次取余和整除即可恢复全部 b。每步都唯一确定一位，最终得到原编码的唯一表示。

输入输出样例

样例输入

```
4 0
2 3 2 5
```

样例输出

```
0 0 0 0
```

输入输出样例

样例输入

```
7 23333
3 5 20 10 4 3 10
```

样例输出

```
2 2 15 7 3 1 0
```

复杂度与边界。时间 $O(n)$，额外空间 $O(1)$，可边读 a 边输出。m 为 0 时每位都为 0；余数不是输出的逆序。

完整参考实现 55_encoding.cpp

```cpp
#include <cstdio>
int main() {
    int n; long long m; scanf("%d%lld", &n, &m);
    for (int i = 0; i < n; i++) {
        int base; scanf("%d", &base);
        printf("%lld%c", m%base, i+1 == n ? '\n' : ' ');
        m /= base;
    }
    return 0;
}
```

2.9 相反数与相邻数对：不重不漏

做题：相反数（真题，第 1 次，2014-03，第 1 题）（CSP201403A）、相邻数对（真题，第 2 次，2014-09，第 1 题）（CSP201409A）。**解答**。这两题数据小，读入数组后枚举 $i < j$ 即可。相反数检查 $a_i + a_j = 0$；相邻数对检查差为 1 或-1。每个无序对只枚举一次。题面保证输入值互不相同；若改成可重复，按值存在性计数便不等于按位置数对计数。

```cpp
int answer = 0;
for (int i = 0; i < n; i++) {
    for (int j = i+1; j < n; j++) {
        if (1LL*a[i]+a[j] == 0) answer++; // Opposite numbers.
        // Neighbor pairs: replace the condition above by
        // 1LL*a[i]-a[j] == 1 || 1LL*a[i]-a[j] == -1
    }
}
```

时间 $O(n^2)$，数组空间 $O(n)$。值域小且不重复时也可用存在性桶：相反数只检查正数 x 与-x，相邻数对只检查 x 与 x+1，这同样只计一次。

2.10 门禁系统与灰度直方图：同一种桶的两种输出

做题：门禁系统（真题，第 3 次，2014-12，第 1 题）（CSP201412A）、灰度直方图（真题，第 22 次，2021-04，第 1 题）（CSP202104A）。门禁题输入 n 及 n 次记录，每条是编号，输出每条编号在当前前缀中的出现次数；原题 $1 \le n \le 1000$，编号在 1 到 n 之间。直方图题输入 n、m、L 和 $n \times m$ 个灰度值，灰度在 0 到 $L-1$ 之间，输出 L 个计数。**解答**。数组 cnt 全 0，每读入一个 x 就 `cnt[x]++`。门禁在更新后立即输出，直方图在全部更新后按 0 到 L-1 输出。

```
int cnt[1005] = {0};
for (int i = 0; i < n; i++) {
    int id; scanf("%d", &id);
    printf("%d%c", ++cnt[id], i+1 == n ? '\n' : ' ');
}
// Histogram core: read all pixels and do cnt[gray]++.
// Then print cnt[0], cnt[1], ..., cnt[L-1], including zeros.
```

每条记录只访问一次，时间线性；存储取决于值域，不是记录数量。多个测试实例时必须重新清零。

2.11 数列分段与分蛋糕：何时增加答案

做题：数列分段（真题，第 5 次，2015-09，第 1 题）（CSP201509A）、分蛋糕（真题，第 10 次，2017-03，第 1 题）（CSP201703A）。数列分段把相邻相等项放在同一段，输入 n 及数组，输出段数；n 至少为 1。**解答**。先有一段，每个 $a_i \neq a_{i-1}$ 的位置开启新段，答案为 1 加变化次数。

```
int segments = 1;
for (int i = 1; i < n; i++) if (a[i] != a[i-1]) segments++;
```

分蛋糕按输入次序累积正重量，达到 k 就交给一个人并从 0 重新累积；最后不足 k 但仍有蛋糕也交给一个人。输入 n、k 和各块重量，输出人数。

```
long long sum = 0; int people = 0;
for (int i = 0; i < n; i++) {
    sum += weight[i];
    if (sum >= k) { people++; sum = 0; }
}
if (sum > 0) people++;
```

达到阈值就分出，符合题目指定流程；若从最大化可分人数理解，正重量条件下延后分出只会消耗更多块，不能增加后续可分段数。两题时间 $O(n)$；读入可流式处理，额外空间 $O(1)$。

2.12 卖菜：首尾与同步更新

做题：卖菜（真题，第 14 次，2018-09，第 1 题）（CSP201809A）。输入至少 2 个原价格；内部新价格是左、中、右原价格之和除以 3 向下取整，两端是两个相邻原价格之和除以 2。输出新价格序列。**解答与关键代码**。全部原值先读入 a，再按规则输出，或者写入新数组 b；不要修改 a 后再算后面的均值。价格非负，整数除法正好完成向下取整。

```
printf("%d", (a[0]+a[1])/2);
for (int i = 1; i+1 < n; i++) printf(" %d", (a[i-1]+a[i]+a[i+1])/3);
printf(" %d\n", (a[n-2]+a[n-1])/2);
```

时间 $O(n)$，空间 $O(n)$。n=2 时内部循环不执行，两个答案都是两项均值。

2.13 报数：数字与轮次分开

做题：报数（真题，第 18 次，2019-12，第 1 题）（CSP201912A）。四人依次轮流报自然数，从 1 开始；7 的倍数或十进制含 7 时跳过并记录当前人的跳过次数。直到成功报出 n 个数，输出四人的跳过次数（每人一行）。**解答**。变量 x 记录当前自然数，done 只统计成功次数；每处理一个 x，轮次都会推进，包括被跳过的数。当前人是 $(x-1) \bmod 4$。

```
bool skip(int x) {
    if (x % 7 == 0) return true;
    while (x) { if (x % 10 == 7) return true; x /= 10; }
    return false;
}
// int cnt[4]={0}, done=0;
// for (int x=1; done<n; x++)
//     if (skip(x)) cnt[(x-1)%4]++; else done++;
```

不要在跳过时停留在同一个人，也不要把 x 直接限制为 n。若实际处理 T 个数，时间 $O(T \log T)$，空间 $O(1)$。

2.14 归一化处理：均值、方差与浮点除法

做题：归一化处理（真题，第 26 次，2022-06，第 1 题）（CSP202206A）。给定 n 个整数，计算均值 $\mu = \frac{1}{n}\sum a_i$、总体方差 $D = \frac{1}{n}\sum (a_i - \mu)^2$，再逐行输出 $(a_i - \mu)/\sqrt{D}$；原题保证方差非零。**解答与关键代码**。使用 double 累加，先求均值，再求平方偏差和。不要误用样本方差的分母 n-1，也不要把平方根漏掉。

```
double sum = 0, variance = 0;
for (int i = 0; i < n; i++) sum += a[i];
double mean = sum/n;
for (int i = 0; i < n; i++) {
    double d = a[i]-mean; variance += d*d;
}
variance /= n;
for (int i = 0; i < n; i++) printf("%.10f\n", (a[i]-mean)/sqrt(variance));
```

需包含 <cmath> 并调用相应 sqrt。时间 $O(n)$，数组空间 $O(n)$。自编变式若允许全部相等，必须规定 D=0 时的输出，不能直接除以 0。

2.15 重复局面：不用 STL 也能比较字符串

做题：重复局面（真题，第 30 次，2023-05，第 1 题）（CSP202305A）。输入 n 个 8×8 字符棋盘，按输入顺序输出当前棋盘在截至此刻的全部棋盘中出现了几次。原题 $n \le 100$。**解答**。每个棋盘压成一个长度 64 的字符数组，末尾加 0；枚举此前及当前所有棋盘，用 strcmp 判断完全相同。n 很小，$O(64n^2)$ 足够，

```
char board[105][65];
// Read each row into char row[9] with scanf("%8s", row),
// then copy its 8 characters to board[i]+8*r.
// Set board[i][64]=0 before strcmp.
for (int i = 0; i < n; i++) {
    int count = 0;
    for (int j = 0; j <= i; j++)
        if (strcmp(board[i], board[j]) == 0) count++;
    printf("%d\n", count);
}
```

包含 <cstring>。比较必须覆盖全部 64 格，不能只用某一个格子或棋子数量作为身份。

2.16 图像旋转

真题 · 第 4 次 CCF-CSP 认证，2015-03，第 1 题。图像旋转（CSP201503A）。**题目与输入输出。**输入一个 $n \times m$ 矩阵，将它逆时针旋转 90° 后输出。第一行 n、m，随后 n 行每行 m 个整数；输出 m 行，每行 n 个整数。$1 \le n, m \le 1000$，元素在 0 到 1000 之间。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 256MB。**读题思路。**圈出“逆时针”和“输出 m 行 n 列”。矩阵不一定是正方形，不能把行列长度混用。百万个 int 约 3.8MiB，可以存整个原矩阵。**做题思路与解答。**原矩阵第 j 列从上到下，会成为新矩阵第 $m-1-j$ 行（0 下标）。因此按列从右到左、每列从上到下输出即可；无需再开一张新矩阵。映射式为 $b[m-1-j][i] = a[i][j]$。每个输入元素恰好输出一次，所以既没有漏项，也没有重复。

输入输出样例

样例输入

```
2 3
1 5 3
3 2 4
```

样例输出

```
3 4
5 2
1 3
```

复杂度与边界。时间 $O(nm)$，空间 $O(nm)$。用 1×3 和 3×1 检查方向；每行最后一个值后输出换行。

完整参考实现 35_rotate.cpp

```cpp
#include <cstdio>
int a[1005][1005];
int main() {
    int n, m; scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &a[i][j]);
    for (int j = m - 1; j >= 0; j--) {
        for (int i = 0; i < n; i++)
            printf("%d%c", a[i][j], i + 1 == n ? '\n' : ' ');
    }
    return 0;
}
```

2.17 数位之和

真题 · 第 6 次 CCF-CSP 认证，2015-12，第 1 题。数位之和（CSP201512A）。**题目与输入输出。**输入一个整数 n，输出其十进制各位数字之和。$0 \le n \le 10^9$。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 256MB。**读题思路。**任务是求数字之和，不是数位数量，也不是把各位拼成新数。输入允许 0；不必为了拆位把整数转换为字符串。**做题思路与解答。**每轮用 $n \bmod 10$ 取个位，再用整数除法 $n/10$ 删除个位；不断累加。每位恰好取一次，得到所求总和。do-while 会对 0 执行一次，结果仍为 0；普通 while 配合总和初值 0 也正确。

输入输出样例

样例输入

```
20151220
```

样例输出

```
13
```

复杂度与边界。若十进制位数为 d，时间 $O(d)$，空间 $O(1)$。10^9 只有 10 位，和最多 81（10^9 本身为 1）；int 足够。

完整参考实现 36_digits.cpp

```cpp
#include <cstdio>
int main() {
    int n, answer = 0; scanf("%d", &n);
    do { answer += n % 10; n /= 10; } while (n != 0);
    printf("%d\n", answer);
    return 0;
}
```

2.18 折点计数

真题 · 第 7 次 CCF-CSP 认证，2016-04，第 1 题。折点计数（CSP201604A）。**题目与输入输出**。有 n 天销售量 $a_1, \ldots, a_n$，相邻两天的销售量不同。一个内部位置的值严格大于左右两项，或严格小于左右两项，称为折点。输入 n 及 n 项销售量，输出折点数。$1 \le n \le 1000$，销售量在 0 到 10000 之间。**评测限制**。曙梦 OJ 当前时限 1 秒，内存 256MB。**读题思路**。折点表示趋势从升变降或从降变升。第一天和最后一天没有两侧邻居，不能计入；不相邻的位置可以取相同值。**做题思路与解答**。从第二项到倒数第二项检查“同时大于左右”或“同时小于左右”。每个内部位置独立判断一次，恰好覆盖全部折点。也可检查相邻差的符号是否不同，但直接写大小关系更直观，且没有乘法溢出的隐患。

输入输出样例

样例输入

```
7
5 4 1 2 3 6 4
```

样例输出

```
2
```

复杂度与边界。时间 $O(n)$，空间 $O(n)$。n 为 1 或 2 时循环不执行，答案为 0。严格单调序列也没有折点。

完整参考实现 37_turns.cpp

```cpp
#include <cstdio>
int a[1005];
int main() {
    int n, answer = 0; scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 1; i + 1 < n; i++) {
        if ((a[i] > a[i-1] && a[i] > a[i+1]) ||
            (a[i] < a[i-1] && a[i] < a[i+1])) answer++;
    }
    printf("%d\n", answer);
    return 0;
}
```

2.19 仓库规划

真题 · 第 32 次 CCF-CSP 认证，2023-12，第 1 题。仓库规划（CSP202312A）。**题目与输入输出。**n 个仓库各有 m 维坐标。只有 j 的每一维都严格大于 i，j 才是 i 的上级；若有多个，取编号最小的 j，否则为 0。输入 n、m，再输入 n 行坐标；按编号顺序输出各仓库上级，一行一个。$1 \le n \le 1000$，$1 \le m \le 10$，各坐标绝对值不超过 10^6。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路。**关注“每一维”“严格大于”“编号最小”。输出不是最近仓库，也不是坐标最小仓库。$n^2 m \le 10^7$，直接枚举候选可行，不必急着排序或使用 STL。**做题思路与解答。**对每个 i，按编号递增枚举 j；逐维检查，一旦出现 $a[j][k] \le a[i][k]$ 便拒绝。第一个全部通过的 j 就是编号最小答案，可以退出候选循环。自身必然无法通过严格比较。遍历所有候选保证无遗漏，升序与首次接受保证最小编号。

输入输出样例

样例输入

```
4 2
0 0
-1 -1
1 2
0 -1
```

样例输出

```
3
1
0
3
```

复杂度与边界。时间 $O(n^2 m)$，空间 $O(nm)$。相等坐标不是上级；无上级时保留初值 0。

完整参考实现 `39_warehouses.cpp`

```cpp
#include <cstdio>
int a[1005][15];
int main() {
    int n, m; scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i++)
        for (int k = 0; k < m; k++) scanf("%d", &a[i][k]);
    for (int i = 1; i <= n; i++) {
        int answer = 0;
        for (int j = 1; j <= n; j++) {
            bool upper = true;
            for (int k = 0; k < m; k++) {
                if (a[j][k] <= a[i][k]) { upper = false; break; }
            }
            if (upper) { answer = j; break; }
        }
        printf("%d\n", answer);
    }
    return 0;
}
```

2.20 统计一组整数（教学例题）

题目。输入 n 个整数，输出总和、最小值、最大值，以及出现次数最多的值及其次数。若多个值并列，输出最小的值。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$。

输入。第一行 n，第二行 n 个整数。**输出**。第一行依次输出总和、最小值、最大值；第二行输出众数及出现次数。

输入输出样例

样例输入

```
6
3 -1 3 2 -1 5
```

样例输出

```
11 -1 5
-1 2
```

解答。总和与极值可边读边更新；最小值不能初始化为 0，否则全正数时会出错。众数作为本例的进阶要求：先调用 C 标准库 qsort 排序，再扫描每个相等值段。按值升序扫描且只在次数严格增加时更新答案，就保留了并列时较小的值。qsort 的比较函数返回负数、0 或正数，表示小于、等于或大于；不要用可能溢出的相减判断。扫描为 $O(n)$，空间为 $O(n)$，总时间由排序主导。C 标准未规定 qsort 的复杂度保证；需要明确的 $O(n \log n)$ 比较次数保证时，可使用第二篇讲解的 sort。初学时先掌握总和与极值，再学习众数部分。

完整参考实现 01_stats.cpp

```cpp
#include <cstdio>
#include <cstdlib>
#include <climits>
long long a[200005];
int compare(const void *p, const void *q) {
    long long x = *(const long long *)p;
    long long y = *(const long long *)q;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}
int main() {
    int n;
    scanf("%d", &n);
    long long sum = 0, mn = LLONG_MAX, mx = LLONG_MIN;
    for (int i = 0; i < n; i++) {
        scanf("%lld", &a[i]); sum += a[i];
        if (a[i] < mn) mn = a[i];
        if (a[i] > mx) mx = a[i];
    }
    qsort(a, n, sizeof(a[0]), compare);
    long long mode = a[0];
    int best = 0, i = 0;
    while (i < n) {
        int j = i;
        while (j < n && a[j] == a[i]) j++;
        if (j - i > best) { best = j - i; mode = a[i]; }
        i = j;
    }
    printf("%lld %lld %lld\n%lld %d\n", sum, mn, mx, mode, best);
    return 0;
}
```

2.21 网格的一轮同步更新（教学例题）

题目。给定一个 $n \times m$ 的 0/1 网格。一次更新后，每个格子的新值由其旧状态的上下左右四邻格决定：值为 1 的邻格至少有两个，则新值为 1，否则为 0。越界邻格不计入。只进行一轮更新，$1 \le n, m \le 500$。

输入。第一行 n, m，随后每行 m 个 0 或 1。**输出**。更新后的网格。

输入输出样例

样例输入

```
2 3
1 0 1
0 1 0
```

样例输出

```
0 1 0
1 0 1
```

推演。左上格的两个有效邻格都是 0，所以变为 0；上方中间格的三个邻格都是 1，所以变为 1。所有计数都使用原数组 a，把答案放进 b。若直接覆盖 a，后面的格子会读到已经变化的数据。

正确性。程序对每格恰好检查四个方向，只有合法坐标参与计数，因此统计值与题意一致；写

入 b 不改变 a，满足同步更新。时间和空间均为 $O(nm)$。

完整参考实现 02_grid.cpp

```cpp
#include <cstdio>
const int N = 505;
int a[N][N], b[N][N];
int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &a[i][j]);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            int count = 0;
            for (int k = 0; k < 4; k++) {
                int x = i + dx[k], y = j + dy[k];
                if (x >= 0 && x < n && y >= 0 && y < m)
                    count += a[x][y];
            }
            b[i][j] = (count >= 2);
        }
    }
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            printf("%d%c", b[i][j], j + 1 == m ? '\n' : ' ');
    return 0;
}
```

3 练习题

练习题型	做题链接
数组排序	P1177 【模板】排序
排序与去重	P1059 明明的随机数

第二部分　第二题：工具与算法

1　基础算法

调用 std::sort、std::max、std::min 时包含 `<algorithm>`；使用 std::vector 时包含 `<vector>`。

1.1　前缀和

1.1.1　一维前缀和

前缀和 S_i 是数组前 i 项的和。数组 a 使用 1 下标，令 $S_0 = 0$，则

$$S_i = S_{i-1} + a_i, \qquad \sum_{i=l}^{r} a_i = S_r - S_{l-1}.$$

右式减去左端点之前的部分，只留下闭区间 $[l, r]$。例如 $a = [3, -2, 7]$，$S = [0, 3, 1, 8]$，第 2 到 3 项的和为 $8 - 3 = 5$。

```
s[0] = 0;
for (int i = 1; i <= n; i++) s[i] = s[i-1] + a[i];
long long range_sum(int l, int r) {
    return s[r] - s[l-1];
}
```

a、s 均至少有 $n + 1$ 格，s 使用 long long。预处理 $O(n)$，每次查询 $O(1)$，空间 $O(n)$；适用于数组不变的多次区间查询。

1.1.2　二维前缀和

$S_{i,j}$ 表示左上角 $(1, 1)$ 到 (i, j) 的元素和。上方与左方前缀相加时，左上区域重复一次，需要减去：

$$S_{i,j} = S_{i-1,j} + S_{i,j-1} - S_{i-1,j-1} + a_{i,j}.$$

闭矩形 $[x_1, x_2] \times [y_1, y_2]$ 的和为

$$S_{x_2,y_2} - S_{x_1-1,y_2} - S_{x_2,y_1-1} + S_{x_1-1,y_1-1}.$$

```
// a, s: global arrays; row 0 and column 0 of s are zero.
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        s[i][j] = s[i-1][j] + s[i][j-1]
                - s[i-1][j-1] + a[i][j];
long long rectangle_sum(int x1, int y1, int x2, int y2) {
    return s[x2][y2] - s[x1-1][y2]
         - s[x2][y1-1] + s[x1-1][y1-1];
}
```

第 0 行、第 0 列提供空前缀，使贴边矩形也能使用同一公式。预处理 $O(nm)$，查询 $O(1)$，空间 $O(nm)$。

1.2 差分

1.2.1 一维差分

差分记录相邻项的变化：$d_i = a_i - a_{i-1}$，其中 $a_0 = 0$。前缀累加 d 可以恢复 a。给闭区间 $[l, r]$ 加 v，只需在 l 开始增加 v，在 $r+1$ 取消这次增加。

```
for (int i = 1; i <= n; i++) d[i] = a[i] - a[i-1];
d[n+1] = 0;
void add_range(int l, int r, long long v) {
    d[l] += v;
    d[r+1] -= v;
}
// After all updates:
long long current = 0;
for (int i = 1; i <= n; i++) {
    current += d[i];
    a[i] = current;
}
```

a、d 使用 long long，$a_0 = 0$，d 至少开 $n+2$ 格。例如 $[3, -2, 7]$ 给 $[2, 3]$ 加 4 后为 $[3, 2, 11]$。每次修改 $O(1)$，q 次修改后统一恢复，总时间 $O(n+q)$，空间 $O(n)$；修改过程中若要查询真实值，需要另计恢复成本。

1.2.2 二维差分

给一个矩形加 v，在左上角开始生效，在右侧和下侧取消，并在右下方补回被取消两次的部分。

```
// d: global long long array, initially zero.
void add_rectangle(int x1, int y1, int x2, int y2,
                   long long v) {
    d[x1][y1] += v;
    d[x2+1][y1] -= v;
    d[x1][y2+1] -= v;
    d[x2+1][y2+1] += v;
}
// After all updates, d[i][j] becomes the accumulated increment.
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        d[i][j] += d[i-1][j] + d[i][j-1] - d[i-1][j-1];
```

矩阵用 1 下标，d 至少有 $(n+2) \times (m+2)$ 格，第 0 行、第 0 列为 0。若原矩阵不是全 0，恢复后将增量加到原值上。q 次修改与恢复共 $O(q+nm)$ 时间、$O(nm)$ 空间。

1.3 二分

1.3.1 有序数组上的边界

二分每次检查中点，排除一半候选。对非降数组 $a[0..n-1]$ 寻找第一个不小于 x 的位置，维护左闭右开区间 $[l, r)$；r 初值为 n，也表示不存在这样的元素。

```
int first_ge(const int a[], int n, int x) {
    int l = 0, r = n;
    while (l < r) {
        int mid = l + (r-l) / 2;
        if (a[mid] < x) l = mid + 1;
        else r = mid;
    }
    return l;
}
```

若 $a_{mid} < x$，mid 及其左侧都不可能是答案；否则 mid 仍可能是第一个，保留它。例如 $[2, 2, 5, 7]$ 查 2 返回 0，查 3 返回 2，查 8 返回 4。检查是否找到 x 时，先判断位置小于 n，再访问数组。时间 $O(\log n)$，额外空间 $O(1)$。

1.3.2 二分答案

若判定 check(x) 随 x 增大由假变真，可以二分第一个可行值。初始闭区间 $[L, R]$ 必须包含答案，且 R 可行。

```
int l = L, r = R;
while (l < r) {
    int mid = l + (r-l) / 2;
    if (check(mid)) r = mid;
    else l = mid + 1;
}
int answer = l;
```

可行时保留 mid 及其左侧，不可行时排除 mid 及其左侧。如果判定方向相反，就要相应调整搜索目标和边界。判定耗时 T 时，总时间为 $O(T \log(R - L + 1))$；中点公式仍要求 $r - l$ 在类型范围内。

1.3.3 实数二分

实数不能用 mid 加 1 排除候选。每轮把区间长度减半；做 k 轮后，误差区间长度为 $(R-L)/2^k$。例如求 $x \geq 0$ 的平方根，在 $[0, \max(1, x)]$ 内寻找使平方达到 x 的边界。

```
double sqrt_binary(double x) {
    double l = 0, r = x > 1 ? x : 1;
    for (int t = 0; t < 80; t++) {
        double mid = l + (r-l) / 2;
        if (mid * mid >= x) r = mid;
        else l = mid;
    }
    return (l+r) / 2;
}
```

本片段取 $0 \leq x \leq 10^{12}$，避免平方超出 double 范围。固定轮数控制理论区间误差，最终精度也受浮点舍入影响。每轮常数操作，时间 $O(k)$、额外空间 $O(1)$。

1.4 双指针

双指针用两个单调移动的位置维护区间或合并两个序列。若两个指针都只前进 n 次，内层循环的总执行次数也是 $O(n)$，而不是 $O(n^2)$。

1.4.1 滑动窗口

对非负数组，右端加入元素不会使和减小，左端删除元素不会使和增大。求和不超过 $K \geq 0$ 的最长连续区间：每次扩展右端，超限时移动左端。

```
int left = 0, best = 0;
long long sum = 0;
for (int right = 0; right < n; right++) {
    sum += a[right];
    while (left <= right && sum > K) sum -= a[left++];
    if (right-left+1 > best) best = right-left+1;
}
```

例如 $[2, 1, 3, 1]$、K=4，前两项构成长度 2 的窗口；加入 3 后删除 2，得到 $[1, 3]$。对每个右端，当前 left 是最早的可行左端，因此得到该右端的最长区间。时间 $O(n)$，额外空间 $O(1)$。有负数时单调性不成立，不能直接使用此规则。

1.4.2 两个有序序列

两个非降数组求公共元素时，小的一侧先前进；它已不可能与另一侧当前或后续更大的值相等。相等时同时前进。

```
int i = 0, j = 0, common = 0;
while (i < n && j < m) {
    if (a[i] < b[j]) i++;
    else if (a[i] > b[j]) j++;
    else { common++; i++; j++; }
}
```

重复值按多重集合交集计数：两边分别出现 u、v 次时贡献 $\min(u, v)$。总移动次数不超过 $n+m$，时间 $O(n+m)$，额外空间 $O(1)$。

1.5 贪心

贪心每一步选择当前最合适的方案，并且不回退。要说明这种选择能保留某个最优解；局部看起来更好，并不自动保证全局最优。

1.5.1 按结束时间选区间

从若干半开区间 $[l_i, r_i)$ 中选最多个互不重叠的区间，允许一个区间在另一个结束时开始。按结束时间递增，选择每个与已选区间不冲突的候选。

```
struct Interval { int l, r; };
bool by_end(const Interval &a, const Interval &b) {
    return a.r < b.r;
}
// a[0..n-1], each interval satisfies l < r.
std::sort(a, a+n, by_end);
int count = 0, last = 0;
bool selected = false;
for (int i = 0; i < n; i++) {
    if (!selected || a[i].l >= last) {
        count++; last = a[i].r; selected = true;
    }
}
```

把某个最优方案中的第一个区间换成结束最早的区间，不会占用更多后续时间，因而不减少可选数量。对后续区间重复这个交换理由，得到贪心正确性。排序 $O(n \log n)$，扫描 $O(n)$，存储 $O(n)$。

1.5.2 中位数与距离和

给定 n 个数，选择 x 最小化 $\sum_{i=1}^{n} |a_i - x|$。排序后将最小与最大、次小与次大配对；每对的距离和在两者之间恒定，超过这个范围就增大。所有配对的最佳范围交于中位数处，所以任选一个中位数即可。

```
std::sort(a, a+n);
long long x = a[n/2], answer = 0;
for (int i = 0; i < n; i++) {
    long long d = (long long)a[i] - x;
    answer += d >= 0 ? d : -d;
}
```

n 为偶数时，两项中位数之间的任意 x 都最优。时间 $O(n \log n)$，存储数组 $O(n)$。

1.6 基础数学

1.6.1 最大公约数

$\gcd(a,b)=\gcd(b,a \bmod b)$：两组数的公因数相同。反复取余，余数变为 0 时，剩余非零数就是最大公约数。

```cpp
long long gcd(long long a, long long b) { // a, b >= 0
    while (b != 0) {
        long long r = a % b;
        a = b; b = r;
    }
    return a;
}
```

至少一个数非零时，时间 $O(\log \max(a,b))$，空间 $O(1)$。正整数的最小公倍数为 $a/\gcd(a,b)\times b$；先除再乘减小中间值，但最终结果仍须在 long long 范围内。

1.6.2 质因数分解

正整数 $x>1$ 可以写为 $\prod p_i^{e_i}$，其中 p_i 是不同质数，e_i 是对应指数。试除到 $\sqrt{x}$，找到因子后反复整除。

```cpp
for (long long p = 2; p <= x / p; p++) {
    if (x % p != 0) continue;
    int exponent = 0;
    while (x % p == 0) { x /= p; exponent++; }
    printf("%lld %d\n", p, exponent);
}
if (x > 1) printf("%lld 1\n", x);
```

每次到达 p 时，更小的质因子已全部除尽，所以能整除的 p 一定是质数。结束后若 $x>1$，它只能是一个剩余质因子。用 $p\le x/p$ 避免 p^2 溢出。最坏时间 $O(\sqrt{x})$，额外空间 $O(1)$。

1.6.3 矩阵乘法与结合顺序

A 为 $n\times k$，B 为 $k\times m$ 时，$C=AB$ 是 $n\times m$ 矩阵，

$$C_{i,j}=\sum_{t=0}^{k-1} A_{i,t}B_{t,j}.$$

```cpp
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++) {
        c[i][j] = 0;
        for (int t = 0; t < k; t++)
            c[i][j] += 1LL * a[i][t] * b[t][j];
    }
```

c 使用 long long 且需能容纳累加结果。时间 $O(nkm)$，结果空间 $O(nm)$。矩阵乘法满足结合律：链式乘法可以改变括号，先计算产生较小中间矩阵的一对，但不能交换矩阵顺序。

1.7 搜索

1.7.1 深度优先搜索

DFS 沿一个分支深入，结束后返回，继续尝试其他分支。状态要包含决定后续选择的信息；递归前修改的共享状态，返回后恢复。下面用 path 保存已选前缀，used 标记哪些数正在这个前缀中，依次选择 1 到 n 的未用数字。

```
// n <= 10; path[10], used[11] are global, initially zero.
void dfs(int depth) {
    if (depth == n) { visit(path, n); return; }
    for (int x = 1; x <= n; x++) {
        if (used[x]) continue;
        path[depth] = x; used[x] = 1;
        dfs(depth+1);
        used[x] = 0;
    }
}
```

调用 dfs(0) 开始。每个分支的有效路径长度是 depth，下一次赋值会覆盖 path[depth]，不必清空；used 必须撤销，否则会阻止其他分支选择 x。共有 $n!$ 个完整排列，若 visit 遍历 n 项，总时间 $O(n \cdot n!)$，数组与递归栈空间 $O(n)$。

1.7.2 基础剪枝

剪枝排除不可能产生所需答案的分支，不能只凭当前结果“不好看”就停止。例如正整数子集求和到 target 时，已选和 sum 超过 target 后不可能恢复；sum 加上所有剩余数仍小于 target 时也不可能达到目标。

```
// rest is the sum of all still available positive numbers.
if (sum > target || sum + rest < target) return;
```

若允许负数，第一个条件不再可靠；每条剪枝都要由剩余选择能产生的范围推出。剪枝减少实际搜索量，一般不改变最坏情况的指数级复杂度。

1.7.3 广度优先搜索

BFS 按距离从近到远逐层扩展，适用于每条边代价相同的最短路。用队列保存已发现但未扩展的顶点，首次入队时记录距离，避免重复入队。

```
// adj[u] stores neighbors; dist and q each have at least n slots.
for (int i = 0; i < n; i++) dist[i] = -1;
int head = 0, tail = 0;
dist[start] = 0; q[tail++] = start;
while (head < tail) {
    int u = q[head++];
    for (int i = 0; i < (int)adj[u].size(); i++) {
        int v = adj[u][i];
        if (dist[v] != -1) continue;
        dist[v] = dist[u] + 1;
        q[tail++] = v;
    }
}
```

顶点编号 0 到 $n-1$，每个顶点最多入队一次。队首距离不大于后方顶点，第一次到达 v 时已

经使用最少边数。不可达顶点保留 -1。邻接表有 n 个顶点、m 条边时，时间 $O(n+m)$，总空间 $O(n+m)$；边权不同时不能用普通 BFS 求最小费用。

1.8 动态规划

动态规划把问题拆成会重复出现的子问题，保存它们的答案。状态说明“这一格代表什么”，转移说明“它由哪些更小状态得到”，初始化给出最小子问题的答案。

1.8.1 递推与初始化

每次走 1 或 2 阶台阶，f_i 表示到达第 i 阶的方案数。最后一步来自 $i-1$ 或 $i-2$，两类方案互不相交，所以 $f_i = f_{i-1} + f_{i-2}$。令 $f_0 = 1$ 表示尚未走的空方案。

```
// n <= 90; f has n+1 slots, all results fit in long long.
f[0] = 1;
for (int i = 1; i <= n; i++) {
    f[i] = f[i-1];
    if (i >= 2) f[i] += f[i-2];
}
```

按 i 递增计算时，右侧状态都已得到。时间 $O(n)$，空间 $O(n)$。

1.8.2 最长上升子序列

子序列可以跳过元素，但不能改变先后顺序。令 f_i 表示以 a_i 结尾的最长严格上升子序列长度：

$$f_i = \max\bigl(1,\ \max_{j<i,\ a_j<a_i} (f_j + 1)\bigr).$$

```
int answer = 0;
for (int i = 0; i < n; i++) {
    f[i] = 1;
    for (int j = 0; j < i; j++)
        if (a[j] < a[i] && f[j]+1 > f[i]) f[i] = f[j]+1;
    if (f[i] > answer) answer = f[i];
}
```

单个元素形成长度 1 的序列；接在较小的前驱后，长度增加 1。答案取所有结尾中的最大值，时间 $O(n^2)$，空间 $O(n)$。

更大数据下，令 tail[k] 保存长度为 $k+1$ 的上升子序列的最小末尾值。末尾越小，越容易接上新元素；tail 按值严格递增。

```
int len = 0;
for (int i = 0; i < n; i++) {
    int l = 0, r = len;
    while (l < r) {
        int mid = l + (r-l) / 2;
        if (tail[mid] < a[i]) l = mid+1;
        else r = mid;
    }
    tail[l] = a[i];
    if (l == len) len++;
}
```

新值替换第一个不小于它的末尾，或在末尾追加。例如 $[3,1,2]$ 对应 tail 依次为 [3]、[1]、[1,2]。相等值只能替换，不能延长严格上升序列。len 为答案；tail 本身未必是一条原数组子序列。时间 $O(n \log n)$，空间 $O(n)$。

1.8.3 最长公共子序列

令 $f_{i,j}$ 表示数组 a 前 i 项与 b 前 j 项的最长公共子序列长度。两个空前缀的答案为 0：

$$f_{i,j} = \begin{cases} f_{i-1,j-1} + 1, & a_i = b_j, \\ \max(f_{i-1,j}, f_{i,j-1}), & a_i \neq b_j. \end{cases}$$

相等时把这个共同元素接到较短前缀的答案后；不等时，两个末尾不能同时匹配，至少舍弃一个。

```
// a, b use 1-based indices; f is global and initially zero.
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        if (a[i] == b[j]) f[i][j] = f[i-1][j-1]+1;
        else f[i][j] = std::max(f[i-1][j], f[i][j-1]);
```

答案为 f[n][m]，时间和空间都是 $O(nm)$。例如 $[1,2,3]$ 与 $[2,1,3]$ 的答案是 2。

1.8.4 滚动数组

转移只依赖上一行时，无需保存全部历史行。LCS 可以只存一行：更新前 f[j] 是上方，更新后的 f[j-1] 是左方，另用 diag 保存左上方。

```
// f[0..m] initially zero; a and b use 1-based indices.
for (int i = 1; i <= n; i++) {
    int diag = 0;
    for (int j = 1; j <= m; j++) {
        int old = f[j];
        if (a[i] == b[j]) f[j] = diag+1;
        else f[j] = std::max(f[j], f[j-1]);
        diag = old;
    }
}
```

空间降为 $O(m)$，时间仍为 $O(nm)$。必须先保存 old，再覆盖 f[j]，否则下一格取不到正确的左上方值。

1.9 背包

背包问题在容量限制下选择物品。以下体积与容量均为正整数，第 i 件物品体积为 v_i、价值为 w_i，容量为 V。容量上限版本令 f[j] 表示总体积不超过 j 的最大价值，允许不选，初值全 0。

1.9.1 01 背包

每件物品最多选一次，转移为 $f_j = \max(f_j, f_{j-v_i} + w_i)$。容量倒序遍历，使右侧仍是处理当前物品之前的状态。

```
// f[0..V] initially zero; v, w use 1-based indices.
for (int i = 1; i <= n; i++)
    for (int j = V; j >= v[i]; j--)
        f[j] = std::max(f[j], f[j-v[i]] + w[i]);
```

答案 f[V]，时间 $O(nV)$，空间 $O(V)$。若改为正序，一件体积 2 的物品更新 f[2] 后，又能由 f[2] 更新 f[4]，等于重复选择同一件。

1.9.2 完全背包

每种物品可以无限选。容量正序遍历，让 f[j-v[i]] 包含本轮已经选过当前物品的方案。

```
for (int i = 1; i <= n; i++)
    for (int j = v[i]; j <= V; j++)
        f[j] = std::max(f[j], f[j-v[i]] + w[i]);
```

初值仍全 0，答案 f[V]。时间 $O(nV)$，空间 $O(V)$。

1.9.3 多重背包

每种物品至多有 c_i 件。把数量拆成 1、2、4、……及剩余数量的若干包，每包按一个 01 物品处理，包的体积和价值都乘以包内件数。

```
for (int i = 1; i <= n; i++) {
    int left = count[i];
    for (long long block = 1; left > 0; block *= 2) {
        int take = (int)std::min(block, (long long)left);
        left -= take;
        long long volume = 1LL * take * v[i];
        long long value = 1LL * take * w[i];
        if (volume > V) continue;
        for (int j = V; j >= volume; j--)
            f[j] = std::max(f[j], f[j-(int)volume] + value);
    }
}
```

例如 13 件拆成 1、2、4、6 件；前 3 包能组合成 0 到 7 件，再加 6 覆盖 6 到 13 件，因此 0 到 13 都能表示。count 使用 int，f 使用 long long；乘积及总价值须在 long long 范围内。时间 $O(V\sum_i \log(c_i+1))$，空间 $O(V)$。

1.9.4 分组背包

每组最多选一件。当前组的所有候选都从上一组的 old 数组转移，避免同组物品相互叠加。

```
// group items are stored in v[g][k], w[g][k], k=0..size[g]-1.
for (int g = 0; g < groups; g++) {
    for (int j = 0; j <= V; j++) old[j] = f[j];
    for (int j = 0; j <= V; j++) {
        f[j] = old[j]; // Select none from this group.
        for (int k = 0; k < size[g]; k++)
            if (v[g][k] <= j)
                f[j] = std::max(f[j], old[j-v[g][k]] + w[g][k]);
    }
}
```

所有组共 n 件物品时，时间 $O(nV)$，空间 $O(V)$；若题目要求每组恰选一件，应删除“不选”的转移，并使用不可达初始化。

1.9.5 恰好装满

若 f[j] 表示总体积恰好 j 的最大价值，只能令 $f_0 = 0$，其他容量初值为不可达。转移前检查来源是否可达。

```
const long long NEG = -(1LL << 60);
f[0] = 0;
for (int j = 1; j <= V; j++) f[j] = NEG;
for (int i = 1; i <= n; i++)
    for (int j = V; j >= v[i]; j--)
        if (f[j-v[i]] != NEG)
            f[j] = std::max(f[j], f[j-v[i]] + w[i]);
```

这里采用 01 背包，假定合法总价值绝对值小于 2^{60}；f[V] 仍为 NEG 表示无法恰好装满。不可达不是价值 0，否则会凭空补足容量。

1.9.6 背包计数

计数状态 ways[j] 表示恰好组成体积 j 的方案数。初值 ways[0]=1 表示空选择，其他为 0，转移使用加法。

```
// Distinct positive coin types; MOD is a positive int.
ways[0] = 1;
for (int j = 1; j <= V; j++) ways[j] = 0;
for (int i = 1; i <= n; i++)
    for (int j = v[i]; j <= V; j++)
        ways[j] = (ways[j] + ways[j-v[i]]) % MOD;
```

ways 使用 long long。物品在外、容量正序，统计不限次数的组合，不区分取物品的先后顺序；若每件至多一次，容量改为倒序。容量在外、枚举最后一种物品在内，则统计有顺序的方案。面额 1、2 组成 3 时，组合为 $\{1,1,1\}$ 与 $\{1,2\}$ 共 2 种，有顺序则还区分 $(1,2)$ 与 $(2,1)$，共 3 种。时间 $O(nV)$，空间 $O(V)$。

2 STL

STL 是 C++ 标准库中的容器与算法。算法头文件为 `<algorithm>`，容器头文件通常与名称相同，例如 `<vector>`、`<map>`；priority_queue 使用 `<queue>`，pair 使用 `<utility>`。以下接口使用 std 前缀，按 C++17 编译。

2.1 区间与迭代器

STL 算法通常处理左闭右开区间 $[first, last)$，包含 first、不包含 last。普通数组的指针和容器的迭代器都可表示区间端点。

```
int a[] = {4, 1, 3, 2};
std::sort(a, a+4);       // All four elements.
std::sort(a+1, a+3);     // Only a[1], a[2].
std::vector<int> v(4, 0);
for (std::vector<int>::iterator it = v.begin();
     it != v.end(); ++it) {
    *it = 1;
}
```

begin() 指向第一个元素，end() 指向末元素后面，不能解引用。空容器满足 begin()==end()。指针和 vector 迭代器支持常数时间的加减、下标与相减；set、map 迭代器只能逐个前进或后退，不能加一个整数。遍历 n 个元素为 $O(n)$。

2.2 sort：排序

sort 按升序排列一个区间，要求随机访问迭代器；普通数组和 vector 可以使用。比较函数返回“a 是否应排在 b 前面”，必须使用严格顺序，相等对象不能互相排在前面。

```
bool greater_int(int a, int b) { return a > b; }
struct Student { int score, id; };
bool by_score(const Student &a, const Student &b) {
    if (a.score != b.score) return a.score > b.score;
    return a.id < b.id;
}
// int a[n]; Student students[n]; arrays already contain data.
std::sort(a, a+n);                  // Ascending.
std::sort(a, a+n, greater_int);     // Descending.
std::sort(students, students+n, by_score);
```

最后一种排序先按成绩降序，再按编号升序。比较器不能写成大于等于；sort 不保证等价对象保留原顺序。n 个元素需要 $O(n \log n)$ 次比较，若单次比较不是常数时间，还要计入比较成本。

2.3 unique：去相邻重复

unique 把相邻重复段压成一个元素，返回新的逻辑末尾，不改变容器实际大小。若要去除所有重复值，先排序，让相同值相邻。

```
int a[] = {3, 1, 3, 2, 2};
std::sort(a, a+5);
int len = (int)(std::unique(a, a+5) - a);
// Only a[0..len-1] is the distinct result: 1, 2, 3.
std::vector<int> v = {3, 1, 3, 2, 2};
std::sort(v.begin(), v.end());
v.erase(std::unique(v.begin(), v.end()), v.end());
```

数组只使用新的 len；vector 再用 erase 删除无效尾段。unique 本身 $O(n)$，排序后去重总时间 $O(n \log n)$。新末尾之后的值不能作为去重结果使用。

2.4 lower_bound：第一个不小于

对非降区间，lower_bound 返回第一个大于等于 x 的位置，全部小于 x 时返回末尾。

```
int a[] = {2, 2, 5, 7};
int x = 3;
int p = (int)(std::lower_bound(a, a+4, x) - a);
bool exists = p < 4 && a[p] == x;
// Here p == 2, exists == false.
```

返回值是位置，不一定指向 x；先判断是否到达末尾。普通数组和 vector 上为 $O(\log n)$ 时间。

2.5 upper_bound：第一个严格大于

upper_bound 返回第一个大于 x 的位置。配合 lower_bound，可以得到 x 的全部出现位置 $[l, r)$。

```
int a[] = {2, 2, 5, 7};
int x = 2;
int l = (int)(std::lower_bound(a, a+4, x) - a);
int r = (int)(std::upper_bound(a, a+4, x) - a);
int count = r-l; // Here [l,r) == [0,2), count == 2.
```

普通数组和 vector 上两次查询共 $O(\log n)$。对 set、map 应调用容器自身的 lower_bound、upper_bound；通用算法在这些迭代器上可能进行线性次数的移动。

2.5.1 离散化

离散化把较大的值映射为有序编号，保留相等关系和大小关系。复制原数组、排序去重，再查每个原值的位置。

```
// a[0..n-1] is unchanged; b and rank each have at least n slots.
for (int i = 0; i < n; i++) b[i] = a[i];
std::sort(b, b+n);
int size = (int)(std::unique(b, b+n) - b);
for (int i = 0; i < n; i++)
    rank[i] = (int)(std::lower_bound(b, b+size, a[i]) - b);
```

例如 $[100, 7, 100]$ 变为编号 $[1, 0, 1]$。编号之差不等于原值之差，不能直接用于距离计算。时间 $O(n \log n)$，空间 $O(n)$。

2.6 vector：可变长数组

vector 连续存储元素，支持下标访问和末尾增删。size() 是实际元素数，capacity() 是已分配容量；reserve 只预留容量，resize 改变元素数。

```
std::vector<int> v;
v.reserve(10);          // size=0; no element may be indexed.
v.push_back(4); v.push_back(7);
v.resize(4, 0);         // v = [4,7,0,0].
v[1] = 9;
v.pop_back();           // v = [4,9,0].
v.insert(v.begin()+1, 5); // v = [4,5,9,0].
v.erase(v.begin()+2);   // v = [4,5,0].
v.assign(3, 2);         // Replace all elements: [2,2,2].
```

下标访问、size、empty、back 为 $O(1)$；push_back 均摊 $O(1)$，发生扩容的一次为 $O(n)$；pop_back 为 $O(1)$。中间插入、删除需要移动后续元素，为 $O(n)$；初始化、assign 和 resize 新增 k 个元素至少处理这 k 项。pop_back、back 要求非空，下标必须小于 size。

扩容会使原迭代器、指针和引用失效；未扩容的插入使插入点及之后的迭代器失效，删除使删除点及之后失效。结构改变后按需要重新取得迭代器。

2.7 string：字符串

string 自动维护长度和存储空间，支持下标、拼接、比较、截取及查找。与 scanf 配合时先读入字符数组，再赋给 string；printf 用 c_str() 取得字符数组形式。

```
char word[101];
scanf("%100s", word);
std::string s = word;
s += "!";
printf("%s\n", s.c_str());
std::string part = s.substr(0, 2); // At most two characters.
std::string::size_type p = s.find("ab");
if (p != std::string::npos) printf("%zu\n", p);
s.insert(0, "X");
s.erase(0, 1);
s.replace(0, 1, "Y");
```

substr(pos,len) 从 pos 截取至多 len 个字符；pos 不能超过 size。find 返回起始下标，失败为 npos；不要把 npos 转成 int 再判断。字符串用 == 判断内容相同，用 < 按字典序比较。

下标访问、size、empty 为 $O(1)$；复制和截取需要处理相应字符，长度 k 时为 $O(k)$。中间插入、删除、替换可能移动后续字符，按字符串及替换内容长度估算。比较最坏要检查整个公共前缀；查找也可能反复比较字符，不能视为常数操作。

2.8 pair：两个值的组合

pair 把两个值保存为 first、second；默认比较先比 first，相等再比 second，适合坐标、距离与编号等二元数据。

```
std::pair<int, int> p = std::make_pair(3, 7);
printf("%d %d\n", p.first, p.second);
std::pair<int, int> a[3] = {{2, 4}, {1, 9}, {2, 3}};
std::sort(a, a+3); // (1,9), (2,3), (2,4).
```

访问成员为 $O(1)$。比较成本取决于成员类型；int 成员比较为 $O(1)$。超过两个有明确含义的属性时，用结构体成员名更容易表达。

2.9 stack：后进先出

栈只从一端访问，最后放入的元素最先取出。push 入栈，top 读取栈顶，pop 删除栈顶。

```
std::stack<int> st;
st.push(3); st.push(5);
if (!st.empty()) {
    int x = st.top(); // x = 5.
    st.pop();         // Leaves 3.
}
```

top、pop 要求非空；pop 不返回元素，要先读取再删除。默认底层为 deque，以上操作以及 size、empty 为 $O(1)$，存储 n 项为 $O(n)$。适合保存括号、待返回的位置或最近的未完成操作。

2.10 queue：先进先出

队列从末尾加入、从头部取出，用于按发现顺序处理对象。

```
std::queue<int> q;
q.push(3); q.push(5);
if (!q.empty()) {
    int x = q.front(); // x = 3.
    q.pop();           // Leaves 5.
}
```

front 读队首，back 读队尾，二者及 pop 要求非空。默认底层为 deque，push、pop、front、back、size、empty 为 $O(1)$，空间 $O(n)$。

2.11 priority_queue：优先队列

优先队列用堆维护当前优先级最高的元素，不保证其他元素整体有序。默认是大根堆，top 为最大值；greater 使它成为小根堆。

```
std::priority_queue<int> large;
std::priority_queue<int, std::vector<int>, std::greater<int> > small;
large.push(4); large.push(9);
int maximum = large.top(); // 9.
large.pop();
small.push(4); small.push(9);
int minimum = small.top(); // 4.
```

greater 还需 `<functional>`。push、pop 为 $O(\log n)$，top、size、empty 为 $O(1)$；top、pop 要求非空。没有直接删除任意元素或按下标访问的接口。

结构体比较函数返回 “a 的优先级低于 b”。函数指针可以作为比较器类型：

```
struct Task { int priority, id; };
bool lower_priority(const Task &a, const Task &b) {
    if (a.priority != b.priority) return a.priority < b.priority;
    return a.id > b.id;
}
std::priority_queue<Task, std::vector<Task>,
                    bool (*)(const Task &, const Task &)> pq(lower_priority);
pq.push(Task{2, 3});
pq.push(Task{2, 1}); // Top has id 1.
```

这里优先级大者先出，并列时编号小者先出。比较仍须严格，不能用大于等于或小于等于。

2.12 set：有序集合

set 按值有序存储，重复插入同一个值只保留一份。适用于动态判重、有序遍历和查找前驱后继。

```
std::set<int> s;
s.insert(3); s.insert(7); s.insert(3);
std::set<int>::iterator it = s.find(3);
if (it != s.end()) s.erase(it);
it = s.lower_bound(5); // First value >= 5.
if (it != s.end()) printf("%d\n", *it); // 7.
```

insert、find、按值 erase、成员 lower_bound 及 upper_bound 为 $O(\log n)$；遍历全部元素为 $O(n)$，空间 $O(n)$。迭代器指向的值不能直接修改。

求严格小于 x 的最大值，先找第一个不小于 x 的元素，再退一步；若已经是 begin，就不存在前驱。

```
std::set<int>::iterator it = s.lower_bound(x);
if (it != s.begin()) { --it; printf("%d\n", *it); }
```

end 不能解引用，begin 不能向前退。插入不使已有迭代器失效，删除只使被删元素的迭代器失效。

2.13 multiset：允许重复的有序集合

multiset 与 set 的有序操作相同，但保留重复值。按值 erase(x) 删除全部 x；只删一个副本时，先 find 再按迭代器删除。

```
std::multiset<int> s;
s.insert(4); s.insert(4); s.insert(7);
std::multiset<int>::iterator it = s.find(4);
if (it != s.end()) s.erase(it); // Deletes only one 4.
int copies = (int)s.count(4);   // 1.
s.erase(4);                    // Deletes every remaining 4.
```

插入、find、成员边界查询为 $O(\log n)$。count(x) 和 erase(x) 为 $O(\log n + k)$，k 是 x 的出现次数；按迭代器删除均摊 $O(1)$，与 find 合用总计 $O(\log n)$。

2.14 map：有序键值映射

map 按键有序，每个键对应一个值。适用于计数、对象编号和按名称保存状态。`m[key]` 在键不存在时会插入默认值；只查询是否存在时，用 find。

```
std::map<int, int> freq;
freq[7]++; freq[3]++; freq[7]++;
std::map<int, int>::iterator it = freq.find(5);
if (it != freq.end()) printf("%d\n", it->second);
for (it = freq.begin(); it != freq.end(); ++it)
    printf("%d %d\n", it->first, it->second);
freq.erase(3);
```

遍历输出按键升序排列，为 3 出现 1 次、7 出现 2 次。first 是键，不能直接修改；second 是值，可以更新。下标、insert、find、按键 erase 为 $O(\log n)$，完整遍历为 $O(n)$，空间 $O(n)$；字符串键还需计入比较字符的成本。

遍历时删除，用 erase 返回的下一个迭代器继续：

```
for (std::map<int, int>::iterator it = freq.begin();
     it != freq.end(); ) {
    if (it->second == 0) it = freq.erase(it);
    else ++it;
}
```

不要删除后再递增已经失效的迭代器。

2.15 unordered_map：哈希键值映射

unordered_map 按哈希值组织键值对，不保证遍历顺序，适合只需按键查询和更新的场景。下标仍会插入不存在的键。

```
std::unordered_map<int, int> freq;
freq.reserve(1000);
freq[7]++;
std::unordered_map<int, int>::iterator it = freq.find(7);
if (it != freq.end()) printf("%d\n", it->second);
freq.erase(7);
```

查找、插入和删除平均 $O(1)$，最坏 $O(n)$；遍历 $O(n)$，空间 $O(n)$。reserve 预留预计元素数，减少扩容次数；重新组织哈希桶会使迭代器失效。字符串键的哈希需要读取字符，长度 L 时至少计入 $O(L)$ 成本。

2.16 unordered_set：哈希集合

unordered_set 只保存不同的值，适用于无需排序的判重和存在性查询。

```
std::unordered_set<int> seen;
seen.reserve(1000);
bool inserted = seen.insert(7).second; // true on first insertion.
bool exists = seen.find(7) != seen.end();
seen.erase(7);
```

insert 返回的 second 表示是否插入了新值。查询、插入、删除平均 $O(1)$，最坏 $O(n)$；遍历 $O(n)$，空间 $O(n)$。它没有按值查找前驱、后继的 lower_bound 或 upper_bound 接口。

3 例题速看

3.1 买菜

真题 · 第 14 次 CCF-CSP 认证，2018-09，第 2 题。买菜（CSP201809B）。**题目与输入输出。**两个人各有 n 段在广场装车的时间，两组区间分别按起点严格递增，且同组区间互不接触、不重叠。求两人同时在广场装车的总时长。输入 n、第一人的 n 行起止时刻、第二人的 n 行起止时刻；输出总时长。$1 \le n \le 2000$，时刻在 1 到 10^6 之间。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 256MB。**读题思路。**计算的是连续时间长度，端点接触不贡献时长，所以交集长度不加 1。原题每组都已排序；读入后无需重复排序。**做题思路与解答。**用 i、j 指向尚未处理的两段。交集为 $[\max(l_i, l_j), \min(r_i, r_j)]$，右端大于左端时累加其差。结束更早的一段不可能与另一组后面的区间重叠，因此移动它；若同时结束，两个都移动。每次安全丢弃已经算完的一段，既不漏交集，也不会重复计数。

输入输出样例

样例输入

```
4
1 3
5 6
9 13
14 15
2 4
5 7
10 11
13 14
```

样例输出

```
3
```

复杂度与边界。两个指针各前进至多 n 次，时间 $O(n)$，数组空间 $O(n)$。样例中正时长交集是 $[2,3]$、$[5,6]$、$[10,11]$，总长 3。

完整参考实现 `40_intervals.cpp`

```cpp
#include <cstdio>
struct Interval { int l, r; } a[2005], b[2005];
int main() {
    int n; scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d%d", &a[i].l, &a[i].r);
    for (int i = 0; i < n; i++) scanf("%d%d", &b[i].l, &b[i].r);
    int i = 0, j = 0, answer = 0;
    while (i < n && j < n) {
        int l = a[i].l > b[j].l ? a[i].l : b[j].l;
        int r = a[i].r < b[j].r ? a[i].r : b[j].r;
        if (l < r) answer += r - l;
        if (a[i].r < b[j].r) i++;
        else if (a[i].r > b[j].r) j++;
        else { i++; j++; }
    }
    printf("%d\n", answer);
    return 0;
}
```

3.2 邻域均值

真题 · 第 22 次 CCF-CSP 认证，2021-04，第 2 题。邻域均值（CSP202104B）。**题目与输入输出**。给出 $n \times n$ 灰度矩阵，各值在 0 到 $L-1$。一个像素的邻域包括与它行、列距离分别不超过 r 的所有合法格子（含自身），越出图像的部分裁掉。统计邻域平均灰度不超过 t 的像素数。输入 n、L、r、t，再输入矩阵；输出计数。$n \le 600$，$1 \le r \le 100$，$2 \le t < L \le 256$。**评测限制**。曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路**。每格对应一个裁剪后的矩形查询；边缘像素分母较小，不能固定用 $(2r+1)^2$。条件是“小于等于”。如果每个像素重算整个邻域，工作量最多约 $600^2 \times 201^2$，应预处理。**做题思路与解答**。建二维前缀和，查询每格邻域矩形总和 S，实际格数为 A。用 $S \le tA$ 判定，无需浮点除法。二维容斥保证求出的和恰好是裁剪矩形的所有像素；整数不等式与平均值条件等价。

输入输出样例

样例输入

```
4 16 1 6
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
```

样例输出

```
7
```

复杂度与边界。预处理和全部查询均为 $O(n^2)$，空间 $O(n^2)$。0 行 0 列作为哨兵；r 大于图像边长时，裁剪仍正确。

完整参考实现 42_neighborhood.cpp

```cpp
#include <cstdio>
#include <algorithm>
long long s[605][605];
int main() {
    int n, L, r, t; scanf("%d%d%d%d", &n, &L, &r, &t);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            int x; scanf("%d", &x);
            s[i][j] = s[i-1][j]+s[i][j-1]-s[i-1][j-1]+x;
        }
    }
    int answer = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            int x1 = std::max(1, i-r), x2 = std::min(n, i+r);
            int y1 = std::max(1, j-r), y2 = std::min(n, j+r);
            long long sum = s[x2][y2]-s[x1-1][y2]
                            -s[x2][y1-1]+s[x1-1][y1-1];
            long long area = 1LL*(x2-x1+1)*(y2-y1+1);
            if (sum <= area*t) answer++;
        }
    }
    printf("%d\n", answer);
    return 0;
}
```

3.3 垦田计划

真题 · 第 29 次 CCF-CSP 认证，2023-03，第 2 题。垦田计划（CSP202303B）。**题目与输入输出**。n 块田同时开垦，第 i 块原耗时 t_i；每投入 c_i 资源能缩短 1 天，但最少需要 k 天。共 m 资源，求全部田完成的最少天数。输入 n、m、k，随后 n 行 t_i, c_i。$n, t_i, c_i \le 10^5, m \le 10^9, 1 \le k \le \min t_i$；输出最少天数。**读题思路**。全部田并行，目标是最大单块耗时，不是耗时总和。每块都不能低于 k，资源需求可能达 10^{15}，累加和乘法用 long long。**做题思路与解答**。固定目标天数 T，要让所有 $t_i \le T$，第 i 块至少投入 $\max(0, t_i - T)c_i$，这些资源需求互不替代，和不超过 m 便可行。T 越大成本越低，可行性为“假、假、真、真”；二分 $[k, \max t_i]$ 中第一个可行值，取下中位数，可行令 hi=mid，不可行令 lo=mid+1。每块的必要投入可同时实现，所以判定充分且必要。

输入输出样例

样例输入

```
4 9 2
6 1
5 1
6 2
7 1
```

样例输出

```
5
```

输入输出样例

样例输入

```
4 30 2
6 1
5 1
6 2
7 1
```

样例输出

```
2
```

复杂度与边界。时间 $O(n \log \max t_i)$，空间 $O(n)$。没有资源时答案为最大原耗时；资源很多时也不能小于 k。

完整参考实现 `56_farming.cpp`

```cpp
#include <cstdio>
int t[100005], c[100005], n; long long budget;
bool possible(int days) {
    long long cost = 0;
    for (int i = 0; i < n; i++) {
        if (t[i] > days) cost += 1LL*(t[i]-days)*c[i];
        if (cost > budget) return false;
    }
    return true;
}
int main() {
    int k, hi = 0; scanf("%d%lld%d", &n, &budget, &k);
    for (int i = 0; i < n; i++) {
        scanf("%d%d", &t[i], &c[i]); if (t[i] > hi) hi = t[i];
    }
    int lo = k;
    while (lo < hi) {
        int mid = lo+(hi-lo)/2;
        if (possible(mid)) hi = mid;
        else lo = mid+1;
    }
    printf("%d\n", lo);
    return 0;
}
```

3.4 相似度计算

真题 · 第 33 次 CCF-CSP 认证，2024-03，第 2 题。相似度计算（CSP202403B）。**题目与输入输出。**输入两篇文章的单词个数 n、m，随后分别输入 n 和 m 个仅含英文字母的单词。忽略大小写并去重，分别输出两个集合的交集大小、并集大小，每个一行。数据范围 $n, m \leq 5 \times 10^5$，每词长度不超过 10。**读题思路。**题名涉及相似度，但不输出相似度小数；只输出两个计数。同篇重复单词只算一次，大小写必须统一。**做题思路与解答。**用哈希表记录每个规范化单词在哪篇出现：位 1 表示第一篇，位 2 表示第二篇。第一篇按位或 1，第二篇按位或 2，重复不会改变状态。表中键数就是并集大小，标记为 3 的键数就是交集大小。哈希表平均查找时间为常数；若用 set 或 map 则有对数因子，适合先练语义后比较效率。

输入输出样例

样例输入

```
3 2
The tHe thE
the THE
```

样例输出

```
1
1
```

输入输出样例

样例输入

```
9 7
Par les soirs bleus dete jirai dans les sentiers
PICOTE PAR LES BLES FOULER LHERBE MENUE
```

样例输出

```
2
13
```

复杂度与边界。设词长上界为 W，平均时间 $O((n+m)W)$，空间与不同词数 u 成正比；哈希冲突严重时性能会退化，不提供最坏常数时间保证。reserve 预留容量降低扩容次数，不能保证没有碰撞。

完整参考实现 57_similarity.cpp

```cpp
#include <cstdio>
#include <string>
#include <unordered_map>
int main() {
    int n, m; scanf("%d%d", &n, &m);
    std::unordered_map<std::string, int> mask;
    mask.max_load_factor(0.7f); mask.reserve(n+m);
    for (int article = 0; article < 2; article++) {
        int count = article == 0 ? n : m;
        for (int i = 0; i < count; i++) {
            char word[11]; scanf("%10s", word);
            for (int j = 0; word[j]; j++)
                if (word[j] >= 'A' && word[j] <= 'Z') word[j] += 'a'-'A';
            mask[std::string(word)] |= article == 0 ? 1 : 2;
        }
    }
    int intersection = 0;
    for (std::unordered_map<std::string, int>::const_iterator it = mask.begin();
         it != mask.end(); ++it) if (it->second == 3) intersection++;
    printf("%d\n%d\n", intersection, (int)mask.size());
    return 0;
}
```

3.5 机器人饲养指南

真题 · 第 37 次 CCF-CSP 认证，2025-03，第 2 题。机器人饲养指南（CSP202503B）。**题目与输入输出。**有 n 个苹果，每天可喂机器人 i 个苹果（$1 \le i \le m$），当天奖励为 A_i，同一种喂法可重复使用。所有苹果都必须喂完，求总奖励最大值。输入 n、m，再输入 $A_1, \ldots, A_m$；输出最优奖励。$1 \le n \le 10^4$，$1 \le m \le 100$，$0 \le A_i \le 10^5$，$A_0 = 0$。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路。**每天选择可以重复，但资源要求是“恰好用完”。奖励既不保证递增，也不保证单价递增，按单日奖励最大贪心没有依据。$nm \le 10^6$，一维 DP 可行。**做题思路与解答。**令 $f[v]$ 表示恰好吃掉 v 个苹果的最大收益。按最后一天吃 i 个苹果分类，$f[v] = \max_{1 \le i \le \min(m,v)}(f[v-i] + A_i)$。$f[0] = 0$，其余先设为负无穷，按 v 从小到大计算。每个方案都有最后一天，且前面的最优方案可以接上这一天，所以转移不漏方案，并保留最大奖励。

输入输出样例

样例输入

```
10 5
1 3 5 3 1
```

样例输出

```
16
```

输入输出样例

样例输入

```
4 3
1 60 100
```

样例输出

```
120
```

复杂度与边界。时间 $O(nm)$，空间 $O(n+m)$。第二组样例中先吃 3 再吃 1 只有 101，而两次吃 2 得 120，直接说明贪心会失败。

完整参考实现 44_feeding.cpp

```cpp
#include <cstdio>
long long f[10005];
int value[105];
int main() {
    int n, m; scanf("%d%d", &n, &m);
    for (int i = 1; i <= m; i++) scanf("%d", &value[i]);
    const long long NEG = -(1LL << 60);
    for (int v = 1; v <= n; v++) f[v] = NEG;
    f[0] = 0;
    for (int v = 1; v <= n; v++) {
        for (int i = 1; i <= m && i <= v; i++) {
            if (f[v-i] != NEG && f[v-i]+value[i] > f[v])
                f[v] = f[v-i]+value[i];
        }
    }
    printf("%lld\n", f[n]);
    return 0;
}
```

3.6 梦境巡查

真题 · 第 36 次 CCF-CSP 认证，2024-12，第 2 题。梦境巡查（CSP202412B）。**题目与输入输出**。输入 n，接着输入 $a_0,\dots,a_n$ 及 $b_1,\dots,b_n$。从 0 号区域出发，走第 i 段消耗 a_i，到区域 i 后得到 b_i；每段出发前能量须够支付。当仅第 i 个补给失效时，求完成整条路线并返回 0 号区域所需的最少初始能量 w_i，输出 n 个结果。$1 \le n \le 10^5$，所有消耗与补给在 0 到 1000 之间。**评测限制**。曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路**。n 次逐段模拟为 $O(n^2)$，规模不允许。缺失 b_i 只影响从区域 i 出发及以后的检查；区域 i 之前不受影响。**做题思路与解答**。设 $d_j=\sum_{t=0}^{j}a_t-\sum_{t=1}^{j}b_t$，表示出发走第 j 段前，对初始能量的下界。缺失第 i 个补给后，$j<i$ 仍需 d_j，$j\ge i$ 则需 d_j+b_i。分别预处理 d 的前缀最大值 P 和后缀最大值 S，得到 $w_i=\max(P_{i-1},S_i+b_i)$。因为 $d_0=a_0\ge 0$，无需额外把负答案截成 0。每个出发检查都满足当且仅当初始值不低于这些下界的最大值，因此这是最小可行值。

输入输出样例

样例输入

```
3
5 5 5 5
0 100 0
```

样例输出

```
10 20 10
```

输入输出样例

样例输入

```
3
9 4 6 2
9 4 6
```

样例输出

```
15 10 9
```

复杂度与边界。时间 $O(n)$，空间 $O(n)$。检查 b 本来就是 0 和巨额补给位于中间的情况。

完整参考实现 45_dream.cpp

```cpp
#include <cstdio>
long long a[100005], b[100005], need[100005];
long long pre[100005], suf[100007];
int main() {
    int n; scanf("%d", &n);
    for (int i = 0; i <= n; i++) scanf("%lld", &a[i]);
    for (int i = 1; i <= n; i++) scanf("%lld", &b[i]);
    need[0] = a[0];
    for (int i = 1; i <= n; i++) need[i] = need[i-1]+a[i]-b[i];
    pre[0] = need[0];
    for (int i = 1; i <= n; i++)
        pre[i] = pre[i-1] > need[i] ? pre[i-1] : need[i];
    suf[n] = need[n];
    for (int i = n-1; i >= 0; i--)
        suf[i] = suf[i+1] > need[i] ? suf[i+1] : need[i];
    for (int i = 1; i <= n; i++) {
        long long answer = pre[i-1] > suf[i]+b[i] ?
                           pre[i-1] : suf[i]+b[i];
        printf("%lld%c", answer, i == n ? '\n' : ' ');
    }
    return 0;
}
```

3.7 机器人宿管指南

真题 · 第 42 次 CCF-CSP 认证，2026-05，第 2 题。机器人宿管指南（CSP202605B）。**题目与输入输出**。输入初始苹果数 n、损耗百分比 k、天数 m。每天先丢掉 $\lceil tk/100 \rceil$ 个坏苹果（t 是前一天剩余量），再给每个机器人一个苹果。求最多能供养多少机器人，要求第 m 天吃完后剩余非负。$1 \le n \le 10^{18}$，$1 \le k \le 5$，$1 \le m \le 2000$；保证答案为不超过 10^9 的正整数。**评测限制**。曙梦 OJ 当前时限 0.5 秒，内存 512MB。**读题思路**。顺序是先变质后食用，百分比向上取整。n 超

过 int 范围，即使 long long 也不宜直接算 $n \times 100$；不能用浮点数近似大整数比例。**做题思路与解答**。人数 x 给定后按天模拟检查。剩余映射为 $\lfloor t(100-k)/100 \rfloor - x$，对 t 单调不减，所以人数越少，全程剩余越多，可行人数形成前缀。二分最后一个可行人数，范围 $[0, \min(10^9, n/m)]$。损耗按 $\lfloor t/100 \rfloor k + \lceil (t \bmod 100)k/100 \rceil$ 计算，各项都在 long long 范围内。

输入输出样例

样例输入

```
1000 1 5
```

样例输出

```
194
```

输入输出样例

样例输入

```
19 5 3
```

样例输出

```
5
```

复杂度与边界。时间 $O(m \log 10^9)$，空间 $O(1)$。0 作为可行哨兵，题面保证最终答案为正。第 m 天可以恰好剩 0，不要求还能吃第 m+1 天。

完整参考实现 46_dorm.cpp

```cpp
#include <cstdio>
long long n; int k, m;
bool enough(long long robots) {
    long long remain = n;
    for (int day = 1; day <= m; day++) {
        // ceil(remain*k/100), without overflowing remain*k.
        long long bad = (remain/100)*k + ((remain%100)*k+99)/100;
        remain -= bad;
        if (remain < robots) return false;
        remain -= robots;
    }
    return true;
}
int main() {
    scanf("%lld%d%d", &n, &k, &m);
    long long lo = 0, hi = n/m;
    if (hi > 1000000000LL) hi = 1000000000LL;
    while (lo < hi) {
        long long mid = lo+(hi-lo+1)/2;
        if (enough(mid)) lo = mid;
        else hi = mid-1;
    }
    printf("%lld\n", lo);
    return 0;
}
```

3.8 字符串变换

真题 · 第 35 次 CCF-CSP 认证，2024-09，第 2 题。字符串变换（CSP202409B）。**题目与输入输出。**字符集合为大小写字母、数字和空格，共 63 种。输入以 # 包围的初始字符串，再给 n 个形如 #xy# 的字符替换规则，未定义的字符保持不变；规则构成单射。随后输入 m 与 m 个 k，每次独立询问初始串连续变换 k 次的结果，以 # 包围输出。$n \le 63, m \le 1000, k \le 10^9$，初始串长不超过 100。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路。**保留空格，不能用%s 读整行；每次查询从原始串开始。每轮替换是同时进行，不能按规则就地多次替换同一个位置。巨大 k 不能逐轮模拟。**做题思路与解答。**预处理 $jump[b][c] = f^{2^b}(c)$，初值为一次变换，递推 $jump[b][c] = jump[b-1][jump[b-1][c]]$。把 k 拆成二进制，对每个字符依次应用对应的跳表即可。每个字符独立变化，函数复合保证跳表恰好等于所需次数。单射还允许按置换环长度取模；这里的倍增实现无需依赖单射，便于统一处理。

输入输出样例

样例输入

```
#Hello World#
6
#HH#
#e #
# r#
#re#
#oa#
#ao#
3
1 2 3
```

样例输出

```
#H llarWaeld#
#HrlloeWo ld#
#Hella Warld#
```

复杂度与边界。字符表用 128 项覆盖 ASCII；时间 $O(128 \log K + m|s| \log K)$，空间 $O(128 \log K + |s|)$。输入的两个 # 是包装符号，不参与变换。

完整参考实现 47_transform.cpp

```cpp
#include <cstdio>
#include <cstring>
char s[105], line[110];
int next_char[128], jump[31][128];
void read_line(char text[], int cap) {
    fgets(text, cap, stdin);
    text[strcspn(text, "\r\n")] = 0;
}
void finish_line() { int c; while ((c=getchar()) != '\n' && c != EOF) {} }
int main() {
    read_line(s, sizeof(s));
    for (int c = 0; c < 128; c++) next_char[c] = c;
    int n; scanf("%d", &n); finish_line();
    for (int i = 0; i < n; i++) {
        read_line(line, sizeof(line));
        next_char[(unsigned char)line[1]] = (unsigned char)line[2];
    }
    for (int c = 0; c < 128; c++) jump[0][c] = next_char[c];
    for (int b = 1; b < 31; b++)
        for (int c = 0; c < 128; c++) jump[b][c] = jump[b-1][jump[b-1][c]];
    int m; scanf("%d", &m);
    while (m--) {
        int k; scanf("%d", &k); putchar('#');
        for (int i = 1; s[i] && s[i] != '#'; i++) {
            int c = (unsigned char)s[i];
            for (int b = 0; b < 31; b++) if ((k >> b) & 1) c = jump[b][c];
            putchar(c);
        }
        puts("#");
    }
    return 0;
}
```

3.9 机器人复健指南

真题 · 第 38 次 CCF-CSP 认证，2025-06，第 2 题。机器人复健指南（CSP202506B）。**题目与输入输出。**输入 n、k 和起点 x、y。在 $n \times n$ 场地内每次按马步跳跃，即坐标变化为 $(\pm 1, \pm 2)$ 或 $(\pm 2, \pm 1)$，共八个方向，不得跳出场地。统计不超过 k 步可到达的不同格数，包含起点。$1 \le n, k \le 100$。输出格数。**读题思路。**图中的八方向是马步，不是上下左右及四个对角相邻格。要求不超过 k 步，且一个格子多次到达只计一次。**做题思路与解答。**BFS 把每个格子作为状态，首次访问得到最短步数。到达第 k 层就不再扩展；每个首次入队的格子给答案加 1，起点先计入。无权边 BFS 按距离递增发现状态，所以所有且仅有最短距离不超过 k 的格子被统计。也能用“步数、位置”的分层可达 DP，再对各层取并集，但 BFS 不用重复访问位置，复杂度更低。

输入输出样例

样例输入

```
4 1
1 1
```

样例输出

```
3
```

输入输出样例

样例输入

```
4 2
1 1
```

样例输出

```
8
```

复杂度与边界。时间 $O(n^2)$，空间 $O(n^2)$，每格最多一次入队。n=1 或 2 时可能完全没有合法马步；答案仍包含起点。

完整参考实现 48_knight.cpp

```cpp
#include <cstdio>
#include <cstring>
struct Point { int x, y; } q[10005];
int dist[105][105];
int dx[8] = {1,1,-1,-1,2,2,-2,-2};
int dy[8] = {2,-2,2,-2,1,-1,1,-1};
int main() {
    int n, k, sx, sy; scanf("%d%d%d%d", &n, &k, &sx, &sy);
    memset(dist, -1, sizeof(dist));
    int head = 0, tail = 0, answer = 1;
    q[tail++] = {sx, sy}; dist[sx][sy] = 0;
    while (head < tail) {
        Point p = q[head++];
        if (dist[p.x][p.y] == k) continue;
        for (int d = 0; d < 8; d++) {
            int x = p.x+dx[d], y = p.y+dy[d];
            if (x < 1 || x > n || y < 1 || y > n || dist[x][y] != -1)
                continue;
            dist[x][y] = dist[p.x][p.y]+1;
            q[tail++] = {x,y}; answer++;
        }
    }
    printf("%d\n", answer);
    return 0;
}
```

3.10 回收站选址

真题 · 第 18 次 CCF-CSP 认证，2019-12，第 2 题。回收站选址（CSP201912B）。**题目与输入输出**。输入 n 个两两不同的垃圾点坐标。只有本身有垃圾且上下左右四邻都存在的点可以选址；得分为四个对角邻点中有垃圾的个数。依次输出得分 0 到 4 的选址数量，各一行。$n \le 1000$，坐标绝对值不超过 10^9。**读题思路**。地图坐标巨大且可负，不能按坐标开完整网格。只有已有垃圾点能成为候选；四个正交邻居是资格条件，四个对角邻居只负责评分。**做题思路与解答**。用 set 保存坐标 pair。枚举每个原点，查询四个正交邻点，全部存在后统计四个对角邻点并增加相应频率。set 的键同时包含横纵坐标，查询精确匹配，所以资格和得分都符合定义。n 很小时也可用数组线性查找作为对照。

输入输出样例

样例输入

```
7
1 2
2 1
0 0
1 1
1 0
2 0
0 1
```

样例输出

```
0
0
1
0
0
```

输入输出样例

样例输入

```
2
0 0
-100000 10
```

样例输出

```
0
0
0
0
0
```

复杂度与边界。时间 $O(n \log n)$，空间 $O(n)$。输入坐标绝对值至多 10^9，加减 1 仍在 int 范围内；n 为 1 时输出五个 0。

完整参考实现 58_recycle.cpp

```cpp
#include <cstdio>
#include <set>
#include <utility>
std::set<std::pair<int,int> > points;
int x[1005], y[1005], answer[5];
bool exists(int x, int y) {
    return points.find(std::make_pair(x,y)) != points.end();
}
int main() {
    int n; scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d%d", &x[i], &y[i]); points.insert(std::make_pair(x[i],y[i]));
    }
    for (int i = 0; i < n; i++) {
        int a = x[i], b = y[i];
        if (!exists(a+1,b) || !exists(a-1,b) ||
            !exists(a,b+1) || !exists(a,b-1)) continue;
        int score = exists(a-1,b-1)+exists(a-1,b+1)
                  +exists(a+1,b-1)+exists(a+1,b+1);
        answer[score]++;
    }
    for (int i = 0; i < 5; i++) printf("%d\n", answer[i]);
    return 0;
}
```

3.11 风险人群筛查

真题 · 第 20 次 CCF-CSP 认证，2020-09，第 2 题。风险人群筛查（在线做题）。**题目与输入输出。**输入 n、k、t 及矩形左下 xl、yd、右上 xr、yu；再输入每人的 t 个记录坐标。含边界的矩形内出现过记录即“经过”；有至少 k 条连续记录都在内即“逗留”。输出经过人数和逗留人数，各一行。数据范围 $n \le 24$，$1 \le k \le t \le 1000$，坐标绝对值不超过 10^6。**读题思路。**只检查采样记录，不插值判断相邻记录间的连线。逗留强调连续；分散的 k 次进入不算。矩形边界也在区域内。**做题思路与解答。**对每人维护曾进入标记、当前连续进入长度 run、曾逗留标记。内点令 run 加 1，外点清 0；run 达到 k 后保留逗留标记，即使后来离开也不取消。每人结束时各标记只加一次计数。

输入输出样例

样例输入

```
1 3 8 0 0 10 10
-1 -1 0 0 0 0 -1 -1 0 0 -1 -1 0 0 0 0
```

样例输出

```
1
0
```

复杂度与边界。时间 $O(nt)$，额外空间 $O(1)$。已经确定逗留后仍须读取剩余坐标，否则下个人输入错位。

完整参考实现 59_risk.cpp

```cpp
#include <cstdio>
int main() {
    int n, k, t, xl, yd, xr, yu;
    scanf("%d%d%d%d%d%d%d", &n, &k, &t, &xl, &yd, &xr, &yu);
    int pass = 0, stay = 0;
    for (int person = 0; person < n; person++) {
        bool touched = false, stayed = false; int run = 0;
        for (int j = 0; j < t; j++) {
            int x, y; scanf("%d%d", &x, &y);
            bool inside = xl <= x && x <= xr && yd <= y && y <= yu;
            if (inside) { touched = true; run++; }
            else run = 0;
            if (run >= k) stayed = true;
        }
        pass += touched; stay += stayed;
    }
    printf("%d\n%d\n", pass, stay);
    return 0;
}
```

3.12 非零段划分

真题 · 第 23 次 CCF-CSP 认证，2021-09，第 2 题。非零段划分（在线做题）。**题目与输入输出。**输入 n 项非负整数 A，选择正整数 p，把所有小于 p 的值变 0，求剩余连续非零段数量的最大值。一个非零段必须不能再向左或向右扩展。$n \le 5\times10^5, A_i \le 10^4$。输出最大段数。**读题思路。**阈值变化时段数不单调，不能二分“段数达到某值”。直接枚举每个 p 并扫描全数组约 5×10^9 次，需利用值域。**做题思路与解答。**每段恰好由一个开头贡献。设 $A_0=0$，位置 i 成为开头当且仅当 $A_{i-1}<p\le A_i$。因此当 $A_{i-1}<A_i$ 时，给阈值区间 $[A_{i-1}+1, A_i]$ 贡献 1，用差分记录；遍历阈值恢复段数并取最大值。每个真实段只有一个开头，条件与开头定义等价，所以统计不重不漏。

输入输出样例

样例输入

```
11
3 1 2 0 0 2 0 4 5 0 2
```

样例输出

```
5
```

输入输出样例

样例输入

```
14
5 1 20 10 10 10 10 15 10 20 1 5 10 15
```

样例输出

```
4
```

复杂度与边界。时间 $O(n+V)$，V=10^4，空间 $O(V)$。数组全 0 时答案 0；为 A_i+1 处预留容量。

完整参考实现 60_nonzero.cpp

```cpp
#include <cstdio>
int delta[10005];
int main() {
    int n, previous = 0; scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        int a; scanf("%d", &a);
        // Position i starts a surviving segment iff previous < p <= a.
        if (previous < a) { delta[previous+1]++; delta[a+1]--; }
        previous = a;
    }
    int segments = 0, answer = 0;
    for (int p = 1; p <= 10000; p++) {
        segments += delta[p]; if (segments > answer) answer = segments;
    }
    printf("%d\n", answer);
    return 0;
}
```

3.13 出行计划

真题 · 第 25 次 CCF-CSP 认证，2022-03，第 2 题。出行计划（在线做题）。**题目与输入输出**。输入计划数 n、查询数 m、等待结果时长 k；随后 n 行出行时刻 t_i 和有效时长 c_i，最后 m 个检测时刻 q。q 检测的结果可从 $q+k$ 到 $q+k+c_i-1$ 进入对应场所。每查询输出符合条件的计划数。$n,m,k \le 10^5$，$t_i,c_i,q \le 2\times 10^5$，均为正整数。**读题思路**。有效时间包括刚获得结果的时刻，末端是有效时长减 1。把“这次检测满足哪些计划”反过来，求“每项计划允许在哪些时刻检测”。**做题思路与解答**。条件 $q+k \le t_i \le q+k+c_i-1$ 等价于 $t_i-c_i+1-k \le q \le t_i-k$。每项计划给这个整数区间加 1，按 q 正数范围裁剪后做差分；一次前缀累加得到每个 q 的答案。等价变形保证区间内恰好计入满足要求的计划。

输入输出样例

样例输入

```
6 2 10
5 24
10 24
11 24
34 24
35 24
35 48
1
2
```

样例输出

```
3
3
```

复杂度与边界。时间 $O(n+m+V)$，V=2×10^5，空间 $O(V)$。右端为负或裁剪后左端大于右端时跳过；r+1 必须有空间。

完整参考实现 61_travel.cpp

```cpp
#include <cstdio>
int d[200005];
int main() {
    int n, m, k; scanf("%d%d%d", &n, &m, &k);
    for (int i = 0; i < n; i++) {
        int t, c; scanf("%d%d", &t, &c);
        int l = t-c+1-k, r = t-k;
        if (l < 1) l = 1;
        if (l <= r) { d[l]++; d[r+1]--; }
    }
    for (int q = 1; q <= 200000; q++) d[q] += d[q-1];
    while (m--) { int q; scanf("%d", &q); printf("%d\n", d[q]); }
    return 0;
}
```

3.14 寻宝！大冒险！

真题 · 第 26 次 CCF-CSP 认证，2022-06，第 2 题。寻宝！大冒险！（CSP202206B）。**题目与输入输出**。地图是坐标 0 到 L 的方格点，共 n 棵树，输入 n、L、S 及 n 个树坐标，再输入 $(S+1)\times(S+1)$ 的 0/1 藏宝图。图的第一维按 S 到 0 输入，第二维按 0 到 S 输入。藏宝图左下格 $B[0][0]=1$。求所有与藏宝图完全匹配且不越地图边界的左下位置数。$n\le 1000, L\le 10^9, S\le 50$，树坐标不重复。**读题思路**。不能按 L 开地图，也不能枚举所有坐标。左下格保证有树，只需枚举 n 棵树为候选。匹配要求 1 与 0 都一致，不能只找到规定的树就忽略多余树。输入首行对应 B[S]，不要按 B[0] 读。**做题思路与解答**。先数藏宝图中的树数 need。对每个候选，拒绝越界窗口；扫描全部地图树，若落入窗口却对应藏宝图 0，立即拒绝，否则统计窗口树数。没有多余树且 count=need 时，一一对应保证所有藏宝图 1 也都有树，其余格为 0，故整个窗口完全匹配。

输入输出样例

样例输入

```
5 100 2
0 0
1 1
2 2
3 3
4 4
0 0 1
0 1 0
1 0 0
```

样例输出

```
3
```

输入输出样例

样例输入

```
5 4 2
0 0
1 1
2 2
3 3
4 4
0 0 0
0 1 0
1 0 0
```

样例输出

```
0
```

复杂度与边界。时间 $O(n^2+S^2)$，空间 $O(n+S^2)$。代码严格按题面 $B[S-i][j]$ 的输入下标存储；不额外转置。

完整参考实现 62_treasure.cpp

```cpp
#include <cstdio>
int x[1005], y[1005], pattern[55][55];
int main() {
    int n, L, S; scanf("%d%d%d", &n, &L, &S);
    for (int i = 0; i < n; i++) scanf("%d%d", &x[i], &y[i]);
    int need = 0;
    for (int row = S; row >= 0; row--)
        for (int col = 0; col <= S; col++) {
            scanf("%d", &pattern[row][col]); need += pattern[row][col];
        }
    int answer = 0;
    for (int i = 0; i < n; i++) {
        if (x[i]+S > L || y[i]+S > L) continue;
        bool ok = true; int count = 0;
        for (int j = 0; j < n; j++) {
            int a = x[j]-x[i], b = y[j]-y[i];
            if (a < 0 || a > S || b < 0 || b > S) continue;
            if (!pattern[a][b]) { ok = false; break; }
            count++;
        }
        if (ok && count == need) answer++;
    }
    printf("%d\n", answer);
    return 0;
}
```

3.15 何以包邮？

真题 · 第 27 次 CCF-CSP 认证，2022-09，第 2 题。何以包邮?（在线做题）。**题目与输入输出**。输入 n、包邮阈值 x，随后 n 本书的正整数价格，每本只能选一次。求总价至少 x 的最小选购总价，保证全部价格和不小于 x。$n\le 30, a_i\le 10^4$，故总价 S 不超过 3×10^5。输出最小总价。**读题思路**。不是花费不超过 x 的最大价值，而是总价至少 x 且最小。2^{30} 枚举太大；利用价格和上界做 01 可达性背包。**做题思路与解答**。reachable[s] 表示用已处理书本能否恰好买到 s 元。初值仅 0

为真，每本价格 a 加入时从 S 向 a 倒序更新 $reachable[s] \mid= reachable[s-a]$，保证当前书只使用一次。全部处理后从 x 向上找首个可达 s，就是满足包邮的最小价格。

输入输出样例

样例输入

```
4 100
20
90
60
60
```

样例输出

```
110
```

输入输出样例

样例输入

```
3 30
15
40
30
```

样例输出

```
30
```

复杂度与边界。时间 $O(nS)$，空间 $O(S)$。重复价格仍代表不同书，不能去重；全部购买可行保证扫描能找到答案。

完整参考实现 63_shipping.cpp

```cpp
#include <cstdio>
int a[35]; bool reachable[300005];
int main() {
    int n, x, total = 0; scanf("%d%d", &n, &x);
    for (int i = 0; i < n; i++) { scanf("%d", &a[i]); total += a[i]; }
    reachable[0] = true;
    for (int i = 0; i < n; i++)
        for (int s = total; s >= a[i]; s--)
            if (reachable[s-a[i]]) reachable[s] = true;
    for (int s = x; s <= total; s++)
        if (reachable[s]) { printf("%d\n", s); break; }
    return 0;
}
```

3.16 数字排序（真题）

第 4 次，2015-03，第 2 题。数字排序（CSP201503B）。**任务**。输入 n 个整数，统计每个不同值的出现次数，按次数从大到小输出“值次数”；次数相同按值从小到大输出。**解答与关键代码**。值域小时先用桶计数，再把非零记录存入结构体数组排序。值域大时可用 map 计数后生成记录。比较函数只表达严格顺序，不能用大于等于。

```cpp
struct Record { int value, count; };
bool less_record(const Record &a, const Record &b) {
    if (a.count != b.count) return a.count > b.count;
    return a.value < b.value;
}
// std::sort(records, records+total, less_record);
```

排序时间 $O(u \log u)$，u 是不同值个数；还需加上计数过程的成本。

3.17 稀疏向量

真题 · 第 19 次 CCF-CSP 认证，2020-06，第 2 题。稀疏向量（CSP202006B）。**题目与输入输出。**长度为 n 的两个向量，只有 a、b 项非零。每个向量的非零项按下标严格递增输入，每项为下标、数值；求内积 $\sum_{i=1}^{n} u_i v_i$。首行 n、a、b，随后 a 行 u 和 b 行 v。$n \le 10^9$，$0 < a, b < n$，$a, b \le 5 \times 10^5$，非零数值绝对值不超过 100。输出内积。**评测限制。**曙梦 OJ 当前时限 2 秒，内存 512MB。**读题思路。**n 很大，但非零项数较小，按 n 开数组会浪费数 GiB。只有相同下标的两项才产生非零乘积，排序信息已经给出。最多约 5×10^9 的绝对值累加，需要 long long。**做题思路与解答。**将每组存成下标和值的结构体数组。下标较小的那项在另一组当前及后续项中不可能匹配，可以丢弃；下标相同则乘积累加并同时前进。每个共同非零位置只遇到一次，其他位置贡献为 0，所以扫描值等于原内积。

输入输出样例

样例输入

```
10 3 4
4 5
7 -3
10 1
1 10
4 20
5 30
7 40
```

样例输出

```
-20
```

复杂度与边界。时间 $O(a+b)$，空间 $O(a+b)$，与完整向量长度 n 无关。若改成映射也能做，但已有有序性时双指针更简单。

完整参考实现 41_sparse.cpp

```cpp
#include <cstdio>
struct Item { int index, value; } a[500005], b[500005];
int main() {
    int n, ca, cb; scanf("%d%d%d", &n, &ca, &cb);
    for (int i = 0; i < ca; i++) scanf("%d%d", &a[i].index, &a[i].value);
    for (int j = 0; j < cb; j++) scanf("%d%d", &b[j].index, &b[j].value);
    long long answer = 0;
    int i = 0, j = 0;
    while (i < ca && j < cb) {
        if (a[i].index < b[j].index) i++;
        else if (a[i].index > b[j].index) j++;
        else { answer += 1LL*a[i].value*b[j].value; i++; j++; }
    }
    printf("%lld\n", answer);
    return 0;
}
```

3.18 因子化简

真题 · 第 32 次 CCF-CSP 认证，2023-12，第 2 题。因子化简（CSP202312B）。**题目与输入输出。**对整数 n 进行质因数分解 $n=\prod p_i^{e_i}$，删除所有指数 $e_i<k$ 的整个因子 $p_i^{e_i}$，保留其余因子，输出结果；全部删去时为 1。输入 q，随后 q 行 n、k；每个结果一行。$1\le q\le 10$，$1<n\le 10^{10}$，$1<k\le 10$。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路。**“删除指数不足的整个质因子幂”不是把每个指数减少 k，也不是只保留 p_i^k。例如 $360=2^3\cdot 3^2\cdot 5$，k=3 时答案为 8。n 超过 int 上界，读入必须用 long long。**做题思路与解答。**从 p=2 开始试除，每找到一个质因子就反复整除，记录完整指数以及对应幂。指数达标才把该幂乘入答案。条件 $p\le n/p$ 避免 p^2 溢出，也利用不断缩小的 n 减少试除。结束后残留大于 1 的数是指数为 1 的质因子；由于本题 k 至少为 2，它不保留。所有质因子都被处理且筛选严格按定义，得到所求乘积。

输入输出样例

样例输入

```
3
2155895064 3
2 2
10000000000 10
```

样例输出

```
2238728
1
10000000000
```

复杂度与边界。最坏时间 $O(q\sqrt{\max n})$，空间 $O(1)$。答案初值为 1，覆盖所有因子被删除的情况。

完整参考实现 43_factor.cpp

```cpp
#include <cstdio>
int main() {
    int q; scanf("%d", &q);
    while (q--) {
        long long n, answer = 1; int k;
        scanf("%lld%d", &n, &k);
        for (long long p = 2; p <= n/p; p++) {
            if (n % p != 0) continue;
            int count = 0; long long power = 1;
            do { n /= p; power *= p; count++; } while (n % p == 0);
            if (count >= k) answer *= power;
        }
        // Remaining prime has exponent 1; k is at least 2.
        printf("%lld\n", answer);
    }
    return 0;
}
```

3.19 画图与矩阵运算（真题）

真题，第 2 次，2014-09，第 2 题。画图（CSP201409B）：每块矩形标记二维差分四角，恢复后统计覆盖数大于 0 的单位格，重叠只计一次。

真题，第 30 次，2023-05，第 2 题。矩阵运算（CSP202305B）：对 QK^TV 先算 K^TV，避免构造 $n \times n$ 中间矩阵；输出还需按题意乘对应行权重。

3.20 日期计算（真题）

真题练习：日期计算：第 5 次，2015-09，第 2 题；用月份天数表消耗年内日序号（CSP201509B）。

日期问题先确定历法。公历闰年的判定是：能被 400 整除，或者能被 4 整除但不能被 100 整除。月份长度用表保存，二月按闰年调整。把日期加一天时，先增加日；若超过当月天数，日改为 1 并增加月；若月变成 13，再进入下一年。每步只有一个明确定义的状态变化。

```cpp
bool leap(int y) {
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
}
int days(int y, int m) {
    int d[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};
    return d[m] + (m == 2 && leap(y));
}
```

3.21 静态区间求和（教学例题）

同类练习（在线做题）：P8218 【深进 1. 例 1】求区间和。

使用一维前缀和；每次查询计算右端前缀减去左端前一项。

输入长度为 n 的数组和 q 次闭区间查询，每次输出区间和。$1 \le n, q \le 2 \times 10^5$，$|a_i| \le 10^9$，保证 $1 \le l \le r \le n$。第一行 n, q，第二行数组，随后 q 行各给出 l, r。

输入输出样例

样例输入

```
5 3
2 -1 3 4 5
1 3
2 4
5 5
```

样例输出

```
4
6
5
```

前缀数组为 $[0,2,1,4,8,13]$，查询 $[2,4]$ 得 $8-2=6$。0 号位置让 $l=1$ 的查询无需特殊判断。使用 long long 存前缀和。总复杂度 $O(n+q)$，空间 $O(n)$。

完整参考实现 05_prefix.cpp

```cpp
#include <cstdio>
const int N = 200005;
long long pre[N];
int main() {
    int n, q;
    scanf("%d%d", &n, &q);
    for (int i = 1; i <= n; i++) {
        long long x;
        scanf("%lld", &x);
        pre[i] = pre[i - 1] + x;
    }
    while (q--) {
        int l, r;
        scanf("%d%d", &l, &r);
        printf("%lld\n", pre[r] - pre[l - 1]);
    }
    return 0;
}
```

3.22 批量区间加法（教学例题）

同类练习（在线做题）：P2367 语文成绩。

差分恢复全部成绩后，原题还要取最小值，而不是输出整个数组。

输入长度为 n 的数组，执行 q 次操作，每次给闭区间 $[l,r]$ 加 v，最后输出整个数组。$1\le n,q\le 2\times 10^5$，初始值和修改值绝对值不超过 10^9。

输入输出样例

样例输入

```
5 2
1 2 3 4 5
2 4 3
1 5 -1
```

样例输出

```
0 4 5 6 4
```

先用初始数组构造差分。所有修改都可叠加，最后做一次前缀累加。总时间 $O(n+q)$，空间 $O(n)$。上例第一轮得到 $[1,5,6,7,5]$，第二轮整体减 1。

完整参考实现 06_difference.cpp

```cpp
#include <cstdio>
const int N = 200005;
long long a[N], d[N];
int main() {
    int n, q;
    scanf("%d%d", &n, &q);
    for (int i = 1; i <= n; i++) {
        scanf("%lld", &a[i]);
        d[i] = a[i] - a[i - 1];
    }
    while (q--) {
        int l, r; long long v;
        scanf("%d%d%lld", &l, &r, &v);
        d[l] += v; d[r + 1] -= v;
    }
    long long x = 0;
    for (int i = 1; i <= n; i++) {
        x += d[i];
        printf("%lld%c", x, i == n ? '\n' : ' ');
    }
    return 0;
}
```

3.23 切木条的最大整数长度（教学例题）

同类练习（在线做题）：P2440 木材加工。

用“能切出的段数不少于所需数量”作判定，找最后一个可行长度。

有 n 根木条，第 i 根长度为 a_i。每根可切成若干长度相同的整段，剩余部分丢弃，不能拼接不同木条。至少要得到 k 段，求最大正整数段长；若连长度 1 都无法满足，输出 0。$1 \le n \le 2 \times 10^5$，$1 \le a_i \le 10^9$，$1 \le k \le 10^{12}$。输入第一行 n、k，第二行各长度。

输入输出样例

样例输入

```
4 7
20 15 10 17
```

样例输出

```
7
```

长度 7 时可得 $2+2+1+2=7$ 段；长度 8 时只有 $2+1+1+2=6$ 段。判定函数为 $\sum_i \lfloor a_i/x \rfloor \geq k$，随 x 增大计数不增。0 作为“无正整数解”的哨兵，不传入判定函数，因此不会除以 0。复杂度 $O(n \log \max a_i)$，空间 $O(n)$。

完整参考实现 07_wood.cpp

```cpp
#include <cstdio>
const int N = 200005;
int n;
long long k, a[N];
int check(long long len) {
    long long count = 0;
    for (int i = 0; i < n; i++) {
        count += a[i] / len;
        if (count >= k) return 1;
    }
    return 0;
}
int main() {
    scanf("%d%lld", &n, &k);
    long long lo = 0, hi = 0;
    for (int i = 0; i < n; i++) {
        scanf("%lld", &a[i]);
        if (a[i] > hi) hi = a[i];
    }
    while (lo < hi) {
        long long mid = lo + (hi - lo + 1) / 2;
        if (check(mid)) lo = mid;
        else hi = mid - 1;
    }
    printf("%lld\n", lo);
    return 0;
}
```

3.24 乘积小于 k 的连续区间个数（教学例题）

给定 n 个**正整数**，统计乘积严格小于 k 的非空连续子数组数量。$1 \leq n \leq 2 \times 10^5$，$1 \leq a_i \leq 10^6$，$0 \leq k \leq 10^9$。输入第一行 n、k，第二行数组。

输入输出样例

样例输入

```
4 100
10 5 2 6
```

样例输出

```
8
```

合法区间包括 4 个单元素、$[10,5]$、$[5,2]$、$[2,6]$、$[5,2,6]$，共 8 个。维护窗口乘积，每加入右端点，就在乘积过大时删除左端点。缩小后，以当前右端点结尾的合法区间有 $right - left + 1$ 个。

所有数至少为 1，因此缩短区间不会使乘积增大，合法左端点形成后缀。若允许负数，这个性质会失效；若允许 0，除法维护也需另外处理。k 不大于 1 时没有合法非空区间。按本题约束，乘入新元素前乘积小于 10^9，乘后小于 10^{15}，long long 足够。时间 $O(n)$，空间 $O(n)$。

完整参考实现 08_window.cpp

```cpp
#include <cstdio>
const int N = 200005;
long long a[N];
int main() {
    int n; long long k;
    scanf("%d%lld", &n, &k);
    for (int i = 0; i < n; i++) scanf("%lld", &a[i]);
    if (k <= 1) { puts("0"); return 0; }
    long long product = 1, answer = 0;
    int left = 0;
    for (int right = 0; right < n; right++) {
        product *= a[right];
        while (left <= right && product >= k) product /= a[left++];
        answer += right - left + 1;
    }
    printf("%lld\n", answer);
    return 0;
}
```

3.25 合并石堆的最小代价（教学例题）

有 n 堆石子，每次可以任意选两堆合并，代价等于两堆石子数之和。求合成一堆的最小总代价。本题**没有相邻限制**。$1 \le n \le 2 \times 10^5$，每堆石子数在 1 到 10^9 之间。输入第一行 n，第二行各堆大小。

输入输出样例

样例输入

```
4
1 2 3 4
```

样例输出

```
19
```

先合并 1 和 2，代价 3；再合并 3 和 3，代价 6；最后合并 4 和 6，代价 10。总计 19。

为什么取最小两堆。合并过程可以看成二叉树，每个原始堆是叶子；总代价等于每个叶子权值

乘以其深度之和。在固定树形中，较小权值放到较深位置不会使代价增大。选择一对最深的兄弟叶子，可通过交换把最小的两个权值放到那里。因此存在一个最优方案先合并最小两堆；把它们缩成一堆后，剩下同类子问题。重复此选择即可。

用小根堆维护所有堆，每次取两个最小值再插入它们的和。时间 $O(n \log n)$，空间 $O(n)$。只有一堆时无需合并，答案为 0。

完整参考实现 09_merge.cpp

```cpp
#include <cstdio>
#include <queue>
#include <vector>
#include <functional>
using namespace std;
int main() {
    int n;
    scanf("%d", &n);
    priority_queue<long long, vector<long long>, greater<long long> > q;
    for (int i = 0; i < n; i++) {
        long long x;
        scanf("%lld", &x); q.push(x);
    }
    long long answer = 0;
    while (q.size() > 1) {
        long long x = q.top(); q.pop();
        long long y = q.top(); q.pop();
        answer += x + y; q.push(x + y);
    }
    printf("%lld\n", answer);
    return 0;
}
```

3.26 按字典序输出全排列（教学例题）

同类练习（在线做题）：P1706 全排列问题。

原题要求每个数占 5 个字符宽度，应使用 `printf("%5d", x)`，不能直接沿用单空格输出。

输入 $1 \le n \le 8$，输出 1 到 n 的所有排列，每行一个，按字典序升序。

输入输出样例

样例输入

```
3
```

样例输出

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

path[0..depth-1] 保存已选的前缀，used[x] 标记 x 是否已在前缀中。每层从小到大尝试尚未使用的数，输出顺序自然为字典序。每个排列对应唯一一条长度 n 的路径，因此不重不漏。输出本身有 $n \cdot n!$ 个数字量级，时间 $O(n \cdot n!)$，递归与标记空间 $O(n)$。

完整参考实现 10_permutations.cpp

```cpp
#include <cstdio>
int n, path[10], used[10];
void dfs(int depth) {
    if (depth == n) {
        for (int i = 0; i < n; i++)
            printf("%d%c", path[i], i + 1 == n ? '\n' : ' ');
        return;
    }
    for (int x = 1; x <= n; x++) {
        if (used[x]) continue;
        used[x] = 1; path[depth] = x;
        dfs(depth + 1);
        used[x] = 0;
    }
}
int main() {
    scanf("%d", &n);
    dfs(0);
    return 0;
}
```

3.27 迷宫最短步数（教学例题）

同类练习（在线做题）：P1443 马的遍历。

将四方向改成马的八种跳法，按原题输出整个距离矩阵；不是只输出终点距离。

输入 $n \times m$ 字符网格，点号为可走格，井号为障碍。每步上下左右移动一格，求左上角到右下角的最少步数；不可达输出-1。$1 \le n, m \le 500$，起终点也可能是障碍。第一行 n、m，随后 n 行长度为 m 的字符串。

输入输出样例

样例输入

```
3 4
....
.##.
....
```

样例输出

```
5
```

解答。所有移动代价相等，使用 BFS。起点距离为 0，邻居距离为当前距离加 1；入队时就记录距离，确保同一格最多入队一次。队列按距离层次扩展，因此首次到达就是最短距离。注意这里的“步数”不包含起点，单格可走迷宫答案为 0。时间与空间均为 $O(nm)$。

完整参考实现 11_bfs.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
const int N = 505;
char g[N][N];
int d[N][N], qx[N * N], qy[N * N];
int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++) {
        scanf("%504s", g[i]);
        fill(d[i], d[i] + m, -1);
    }
    int head = 0, tail = 0;
    if (g[0][0] == '.') {
        d[0][0] = 0; qx[tail] = 0; qy[tail++] = 0;
    }
    while (head < tail) {
        int x = qx[head], y = qy[head]; head++;
        for (int k = 0; k < 4; k++) {
            int a = x + dx[k], b = y + dy[k];
            if (a < 0 || a >= n || b < 0 || b >= m) continue;
            if (g[a][b] == '#' || d[a][b] != -1) continue;
            d[a][b] = d[x][y] + 1;
            qx[tail] = a; qy[tail++] = b;
        }
    }
    printf("%d\n", d[n - 1][m - 1]);
    return 0;
}
```

3.28 最长严格上升子序列（教学例题）

同类练习（在线做题）：B3637 最长上升子序列。

先掌握以当前位置结尾的状态，再根据数据范围选择二次或对数优化。

输入 n 个整数，选择一些位置并保持原有顺序，使选出的值严格递增，求最大长度。$1 \le n \le 5000$，值的绝对值不超过 10^9。子序列可以不连续。第一行 n，第二行数组。

输入输出样例

样例输入

```
6
3 1 2 2 5 4
```

样例输出

```
3
```

可取 $[1,2,5]$ 或 $[1,2,4]$。设 f[i] 为**以第 i 个元素结尾**的最长长度，

$$f[i] = 1 + \max\{f[j] : j < i,\ a_j < a_i\},$$

没有合法 j 时 f[i]=1。任何以 i 结尾的较长方案，倒数第二个位置必然是一个合法 j；反过来，把 a[i] 接到该 j 的方案后面也合法。因此枚举所有前驱就覆盖全部可能。答案是所有 f[i] 的最大值，不一定是 f[n]。时间 $O(n^2)$，空间 $O(n)$。

完整参考实现 12_lis.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
const int N = 5005;
int a[N], f[N];
int main() {
    int n, answer = 0;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 0; i < n; i++) {
        f[i] = 1;
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) f[i] = max(f[i], f[j] + 1);
        answer = max(answer, f[i]);
    }
    printf("%d\n", answer);
    return 0;
}
```

3.29 非空最大子段和（教学例题）

同类练习（在线做题）：P1115 最大子段和。

必须选择非空连续区间，检查全负数组。

输入 n 个整数，求一个**连续且非空**区间的最大和。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$。

输入输出样例

样例输入

```
5
-2 3 -1 4 -5
```

样例输出

```
6
```

设 ending 为以当前位置结尾的非空区间最大和。若使用当前数 x，则最优区间要么从 x 重新开始，要么延长前一位置的最优区间：$ending = \max(x, ending + x)$。全局答案记录所有位置的最大 ending。由于要求非空，初始化为第一个元素，不能初始化为 0。时间 $O(n)$，额外空间 $O(1)$。

完整参考实现 13_maxsum.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
int main() {
    int n; long long x;
    scanf("%d%lld", &n, &x);
    long long ending = x, answer = x;
    for (int i = 1; i < n; i++) {
        scanf("%lld", &x);
        ending = max(x, ending + x);
        answer = max(answer, ending);
    }
    printf("%lld\n", answer);
    return 0;
}
```

3.30 数字三角形的最大路径和（教学例题）

输入 n 行数字，第 i 行有 i 个数。从顶点出发，每次走到下一行的正下方或右下方，求到最后一行的最大路径和。$1 \le n \le 1000$，每个数的绝对值不超过 10^6。第一行 n，随后各行给出三角形。

输入输出样例

样例输入

```
3
2
-1 3
4 -2 1
```

样例输出

```
6
```

二维状态 f[i][j] 表示到第 i 行第 j 个位置的最优值，合法前驱只有 $(i-1,j-1)$ 与 $(i-1,j)$。初值为顶点，非法位置为负无穷。转移

$$f[i][j] = a[i][j] + \max(f[i-1][j-1], f[i-1][j]).$$

上例路径 $2,3,1$ 得 6。因为每行只依赖前一行，可压缩成一维。**从右向左更新**才能确保 f[j-1] 尚未被本行覆盖。代码设置 f[1]=0 作为顶点前的虚拟起点，其他位置为负无穷。时间 $O(n^2)$，额外空间 $O(n)$。

完整参考实现 14_triangle.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
const int N = 1005;
const long long NEG = -(1LL << 60);
long long f[N], row[N];
int main() {
    int n;
    scanf("%d", &n);
    fill(f, f + n + 1, NEG); f[1] = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) scanf("%lld", &row[j]);
        for (int j = i; j >= 1; j--)
            f[j] = max(f[j], f[j - 1]) + row[j];
    }
    printf("%lld\n", *max_element(f + 1, f + n + 1));
    return 0;
}
```

3.31 每件物品最多选一次（教学例题）

同类练习（在线做题）：P1048 采药。

时间是容量，药材价值是收益；一维转移按容量逆序。

有 n 件物品，每件体积 c、价值 w，容量为 V，求总体积不超过 V 的最大总价值。$1 \le n \le 200$，$0 \le V \le 5000$，$1 \le c \le 5000$，$0 \le w \le 10^9$。输入第一行 n、V，后面每行 c、w。

输入输出样例

样例输入

```
3 5
2 3
3 4
4 5
```

样例输出

```
7
```

令 f[i][v] 为只考虑前 i 件物品、容量不超过 v 的最大价值。第 i 件选或不选：

$$f[i][v] = \max(f[i-1][v],\ f[i-1][v-c_i] + w_i),$$

第二项只在 $v \geq c_i$ 时存在。若压缩掉 i 维，容量必须从大到小更新，使 f[v-c] 仍是上一层状态，避免同一件被重复选。复杂度 $O(nV)$，空间 $O(V)$。

完整参考实现 15_knapsack01.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
long long f[5005];
int main() {
    int n, capacity;
    scanf("%d%d", &n, &capacity);
    for (int i = 0; i < n; i++) {
        int cost; long long value;
        scanf("%d%lld", &cost, &value);
        for (int v = capacity; v >= cost; v--)
            f[v] = max(f[v], f[v - cost] + value);
    }
    printf("%lld\n", f[capacity]);
    return 0;
}
```

3.32 每种物品可以无限选择（教学例题）

同类练习（在线做题）：P1616 疯狂的采药。

每种可重复选取，按容量正序转移。

输入格式与上一题相同，但现在 n 表示物品种类，每种可以选任意非负整数件。仍要求体积不超过 V、最大化价值，且体积必须为正。采用同样的约束。

输入输出样例

样例输入

```
2 7
2 3
3 4
```

样例输出

```
10
```

选择体积 2 的两件和体积 3 的一件，价值 10。二维转移的选取项变为 $f[i][v-c_i]+w_i$，它允许当前种类继续使用。因此一维容量循环从小到大，使 f[v-c] 已经包含本种物品。时间 $O(nV)$，空间 $O(V)$。

完整参考实现 16_knapsack_complete.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
long long f[5005];
int main() {
    int n, capacity;
    scanf("%d%d", &n, &capacity);
    for (int i = 0; i < n; i++) {
        int cost; long long value;
        scanf("%d%lld", &cost, &value);
        for (int v = cost; v <= capacity; v++)
            f[v] = max(f[v], f[v - cost] + value);
    }
    printf("%lld\n", f[capacity]);
    return 0;
}
```

3.33 无限硬币的组合数（教学例题）

有 n 种**互不相同的正整数面额**，每种无限个，问恰好凑出 target 的组合数，硬币选择顺序不算不同方案。输出对 10^9+7 取模的结果。$1 \le n \le 100$，$0 \le target \le 10^4$，面额不超过 10^4。输入第一行 n、target，第二行所有面额。

输入输出样例

样例输入

```
3 5
1 2 5
```

样例输出

```
4
```

四种为 $1+1+1+1+1$、$1+1+1+2$、$1+2+2$、5。令 f[v] 为已处理面额能凑出的组合数。先枚举面额，再正序枚举金额，确保每种组合按固定面额顺序生成一次。f[0]=1 是构造非空方案的起点。时间 $O(n \cdot target)$，空间 $O(target)$。

完整参考实现 17_coin_count.cpp

```cpp
#include <cstdio>
const int MOD = 1000000007;
int f[10005];
int main() {
    int n, target;
    scanf("%d%d", &n, &target);
    f[0] = 1;
    for (int i = 0; i < n; i++) {
        int c;
        scanf("%d", &c);
        for (int v = c; v <= target; v++) f[v] = (f[v] + f[v - c]) % MOD;
    }
    printf("%d\n", f[target]);
    return 0;
}
```

3.34 每组最多选一件（教学例题）

同类练习（在线做题）：P1757 通天之分组背包。

原题输入包含物品的组号，应先归组，再按整组转移。

有 g 组物品，每组最多选一件，也可以不选。求容量 V 内的最大价值。$1 \le g \le 100, 0 \le V \le 2000$，每组 1 到 20 件，体积为正、价值非负。输入第一行 g、V，随后每组先给物品数 k，再给 k 对体积、价值。

输入输出样例

样例输入

```
2 5
2
2 3
3 5
2
2 4
4 7
```

样例输出

```
9
```

第一组选体积 3 价值 5，第二组选体积 2 价值 4。对每组保留旧数组 f，所有选择都从旧状态转移到 next_f，next_f 初始复制 f 表示不选。这种写法直接保证同一组至多贡献一件，比混用循环顺序更容易检查。若总物品数为 M，时间 $O(MV)$，空间 $O(V)$。

完整参考实现 18_group.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
long long f[2005], next_f[2005], value[25];
int cost[25];
int main() {
    int groups, capacity;
    scanf("%d%d", &groups, &capacity);
    for (int g = 0; g < groups; g++) {
        int k;
        scanf("%d", &k);
        for (int i = 0; i < k; i++) scanf("%d%lld", &cost[i], &value[i]);
        for (int v = 0; v <= capacity; v++) {
            next_f[v] = f[v];
            for (int i = 0; i < k; i++)
                if (v >= cost[i])
                    next_f[v] = max(next_f[v], f[v - cost[i]] + value[i]);
        }
        for (int v = 0; v <= capacity; v++) f[v] = next_f[v];
    }
    printf("%lld\n", f[capacity]);
    return 0;
}
```

3.35 按成绩和编号排序（教学例题）

题目。输入 n 名学生的编号和成绩，成绩降序，相同成绩按编号字典序升序。编号是不含空格的字符串，长度 1 到 30 且互不相同；$1 \le n \le 2 \times 10^5$，成绩 0 到 100。首行 n，随后每行编号和成绩，输出排序后的记录。

输入输出样例

样例输入

```
4
b02 90
a01 95
a02 90
a03 90
```

样例输出

```
a01 95
a02 90
a03 90
b02 90
```

解答。结构体数组保存记录，cmp 先判断成绩，再用 strcmp 判断编号。编号数组容量 31，多留一格保存字符串结束符。%30s 限制一次最多读入 30 个字符。比较次数为 $O(n \log n)$，每次字符串比较最多扫描 30 个字符，记录存储为 $O(n)$。

完整参考实现 03_rank.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <algorithm>
using namespace std;
const int N = 200005;
struct Student { char id[31]; int score; };
Student a[N];
bool cmp(Student x, Student y) {
    if (x.score != y.score) return x.score > y.score;
    return strcmp(x.id, y.id) < 0;
}
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%30s%d", a[i].id, &a[i].score);
    sort(a, a + n, cmp);
    for (int i = 0; i < n; i++) printf("%s %d\n", a[i].id, a[i].score);
    return 0;
}
```

3.36 有序去重（教学例题）

题目。输入 n 个整数，$1 \le n \le 2 \times 10^5$、$|a_i| \le 10^9$。第一行输出不同值数量，第二行按升序输出这些值。

输入输出样例

样例输入

```
6
3 1 3 2 2 1
```

样例输出

```
3
1 2 3
```

排序后每个等值段保留一个即可，时间 $O(n \log n)$。

同类练习（在线做题）：P1059 明明的随机数。

该原题同样要求去重后升序输出，提交时按原题范围与格式读写。

完整参考实现 27_distinct.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
int a[200005];
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    sort(a, a + n);
    int m = unique(a, a + n) - a;
    printf("%d\n", m);
    for (int i = 0; i < m; i++)
        printf("%d%c", a[i], i + 1 == m ? '\n' : ' ');
    return 0;
}
```

3.37 追加和撤销（教学例题）

题目。首行 q，$1 \le q \le 10^5$；随后每条命令为 ADD x、UNDO 或 PRINT，$|x| \le 10^9$。ADD 末尾追加，UNDO 删除末尾一项，空时忽略；PRINT 输出当前序列，空时输出 EMPTY。保证全部 PRINT 实际输出的元素数不超过 2×10^5。

输入输出样例

样例输入

```
7
UNDO
ADD 4
ADD 9
PRINT
UNDO
PRINT
UNDO
```

样例输出

```
4 9
4
```

每次修改仅涉及末尾，vector 适合保存操作结果。时间为 $O(q+S)$ 的均摊量级，S 为输出元素总数；没有对 S 的约束时，反复 PRINT 的输出本身也可能很大。

完整参考实现 33_undo.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <vector>
using namespace std;
int main() {
    int q;
    scanf("%d", &q);
    vector<int> a;
    while (q--) {
        char op[10]; scanf("%9s", op);
        if (strcmp(op, "ADD") == 0) {
            int x; scanf("%d", &x); a.push_back(x);
        } else if (strcmp(op, "UNDO") == 0) {
            if (!a.empty()) a.pop_back();
        } else {
            if (a.empty()) puts("EMPTY");
            else {
                for (int i = 0; i < (int)a.size(); i++)
                    printf("%d%c", a[i], i + 1 == (int)a.size() ? '\n' : ' ');
            }
        }
    }
    return 0;
}
```

3.38 从左向右替换（教学例题）

题目。输入三行 s、p、t，各行长度不超过 1000，允许空格；s 和 t 可空，p 非空。按原串 s 从左到右查找 p，每次选取最靠左的不重叠出现，将其替换为 t。新插入的内容不再次参与匹配，输出结果。例如 s 为 aaaa、p 为 aa、t 为 b，结果 bb；s 为 aaa 时结果 ba。

输入输出样例

样例输入

```
one two one
one
X
```

样例输出

```
X two X
```

用 start 标记原串尚未处理的位置。找到后先追加未匹配片段，再追加 t，随后前进 p 的长度。始终在原串上 find，可避免 t 包含 p 时重复替换甚至死循环。答案最长为约 10^6 字符；朴素匹配可用 $O(|s||p|+|answer|)$ 估计，具体 string 查找实现可能优化，本题规模不依赖这种优化。

完整参考实现 34_replace.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <string>
using namespace std;
char line[1005], pattern[1005], replacement[1005];
void read_line(char s[]) {
    if (fgets(s, 1005, stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
}
int main() {
    read_line(line); read_line(pattern); read_line(replacement);
    string s = line, p = pattern, t = replacement, answer;
    string::size_type start = 0;
    while (true) {
        string::size_type pos = s.find(p, start);
        if (pos == string::npos) break;
        answer += s.substr(start, pos - start);
        answer += t;
        start = pos + p.size();
    }
    answer += s.substr(start);
    printf("%s\n", answer.c_str());
    return 0;
}
```

3.39 三类括号匹配（教学例题）

题目。输入一行仅由 ()[]{} 组成的字符串，长度 0 到 10^5；空串合法。若每个右括号能与最近尚未匹配的同类左括号配对，输出 YES，否则 NO。

输入输出样例 1

样例输入

```
([]){}
```

样例输出

```
YES
```

输入输出样例 2

样例输入

```
([)]
```

样例输出

```
NO
```

最近出现的未匹配左括号必须先配对，所以使用栈。扫描右括号时先检查非空，再检查类型；扫描结束还必须检查栈是否为空。每个字符最多入栈出栈各一次，时间和最坏空间均为 $O(n)$。只比较左右括号总数会误判顺序或嵌套类型。

完整参考实现 28_brackets.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <stack>
using namespace std;
char s[100005];
int match(char left, char right) {
    return (left == '(' && right == ')') ||
           (left == '[' && right == ']') ||
           (left == '{' && right == '}');
}
int main() {
    if (fgets(s, sizeof(s), stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
    stack<char> st;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty() || !match(st.top(), c)) {
                puts("NO"); return 0;
            }
            st.pop();
        }
    }
    puts(st.empty() ? "YES" : "NO");
    return 0;
}
```

3.40 排队取号（教学例题）

题目。首行 n，$1 \le n \le 10^5$；随后每行 PUSH x 或 POP，编号 x 为 int 范围内整数。PUSH 入队；POP 输出并删除队首，空队列输出 EMPTY。

输入输出样例

样例输入

```
6
POP
PUSH 5
PUSH 2
POP
POP
POP
```

样例输出

```
EMPTY
5
2
EMPTY
```

维护进入顺序即可。每条命令 $O(1)$，总时间 $O(n)$，最多保存 n 个编号。

完整参考实现 29_queue.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <queue>
using namespace std;
int main() {
    int n;
    scanf("%d", &n);
    queue<int> q;
    while (n--) {
        char op[10];
        scanf("%9s", op);
        if (strcmp(op, "PUSH") == 0) {
            int x; scanf("%d", &x); q.push(x);
        } else {
            if (q.empty()) puts("EMPTY");
            else { printf("%d\n", q.front()); q.pop(); }
        }
    }
    return 0;
}
```

3.41 保留最大的 k 个数（教学例题）

题目。输入 n、k 及 n 个整数，$1 \le k \le n \le 2 \times 10^5$，数值在 int 范围内。重复值按不同出现计数，按降序输出最大的 k 个数。

输入输出样例

样例输入

```
5 3
4 1 9 9 2
```

样例输出

```
9 9 4
```

维护最多 k 个元素的小根堆。每读入一个数就入堆，超过 k 个时删除最小值；留下的恰好是已读前缀中最大的 k 个。反复弹出得到升序，最后逆转。时间 $O(n\log(k+1)+k)$，额外空间 $O(k)$。若 n 很小且一次读完，直接 sort 也是合理选择。

完整参考实现 32_heap.cpp

```cpp
#include <cstdio>
#include <queue>
#include <vector>
#include <algorithm>
#include <functional>
using namespace std;
int main() {
    int n, k;
    scanf("%d%d", &n, &k);
    priority_queue<int, vector<int>, greater<int> > q;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x); q.push(x);
        if ((int)q.size() > k) q.pop();
    }
    vector<int> a;
    while (!q.empty()) { a.push_back(q.top()); q.pop(); }
    reverse(a.begin(), a.end());
    for (int i = 0; i < (int)a.size(); i++)
        printf("%d%c", a[i], i + 1 == (int)a.size() ? '\n' : ' ');
    return 0;
}
```

3.42 动态候选值（教学例题）

题目。首行 q，$1 \le q \le 2\times 10^5$；随后每行 ADD x、DEL x 或 GE x，$|x| \le 10^9$。ADD 加入一个副本，DEL 删除一个副本，不存在则忽略；GE 输出当前最小的、不小于 x 的值，不存在输出 NONE。

输入输出样例

样例输入

```
8
ADD 5
ADD 5
ADD 9
DEL 5
GE 5
DEL 5
GE 5
GE 10
```

样例输出

```
5
9
NONE
```

使用 multiset 保留重复值；DEL 采用 find 加 erase 迭代器；GE 用成员 lower_bound。每条操作至多 $O(\log(q+1))$，空间 $O(q)$。

完整参考实现 `30_neighbors.cpp`

```cpp
#include <cstdio>
#include <cstring>
#include <set>
using namespace std;
int main() {
    int q;
    scanf("%d", &q);
    multiset<int> s;
    while (q--) {
        char op[10]; int x;
        scanf("%9s%d", op, &x);
        if (strcmp(op, "ADD") == 0) s.insert(x);
        else if (strcmp(op, "DEL") == 0) {
            multiset<int>::iterator it = s.find(x);
            if (it != s.end()) s.erase(it);
        } else {
            multiset<int>::iterator it = s.lower_bound(x);
            if (it == s.end()) puts("NONE");
            else printf("%d\n", *it);
        }
    }
    return 0;
}
```

3.43 单词计数（教学例题）

题目。输入 n 及 n 个仅含小写字母的单词，每词长度 1 到 30，$1 \le n \le 10^5$。按字典序输出每个不同单词及次数。

输入输出样例

样例输入

```
5
pear apple pear banana apple
```

样例输出

```
apple 2
banana 1
pear 2
```

读入时对键计数；map 已经按键排列，最后顺序遍历即可，不需要再 sort。若有 m 个不同单词，计数约需 $O(30n\log(m+1))$ 时间，存储约 $O(30m)$。

完整参考实现 `31_frequency.cpp`

```cpp
#include <cstdio>
#include <map>
#include <string>
using namespace std;
int main() {
    int n;
    scanf("%d", &n);
    map<string, int> count;
    for (int i = 0; i < n; i++) {
        char word[31];
        scanf("%30s", word); count[word]++;
    }
    map<string, int>::iterator it;
    for (it = count.begin(); it != count.end(); ++it)
        printf("%s %d\n", it->first.c_str(), it->second);
    return 0;
}
```

3.44 B3693：数列前缀和 4

同类练习（在线做题）：B3693 数列前缀和 4。

题意。多组非负整数矩阵，每次求矩形和模 2^{64}，每组输出所有查询答案的异或和。$n, m \le 1000$，累计矩阵项数、查询数各不超过 10^6。**解答**。使用 unsigned long long 建立二维前缀和。C++ 无符号加减自然按 2^{64} 回绕，正好实现题目取模；不能改用有符号溢出。每组清零第 0 行和第 0 列，正文符号求和公式其余部分相同。$O(nm+q)$。下面给完整程序。**自测样例（按原题规则构造）**。

输入输出样例

样例输入

```
2
1 2 2
18446744073709551615 1
1 1 1 2
1 1 1 1
1 1 1
0
1 1 1 1
```

样例输出

```
18446744073709551615
0
```

完整参考实现 68_unsigned_prefix.cpp

```cpp
#include <cstdio>
unsigned long long pre[1005][1005];
int main() {
    int tests;
    scanf("%d", &tests);
    while (tests--) {
        int n, m, q;
        scanf("%d%d%d", &n, &m, &q);
        for (int i = 0; i <= n; i++) pre[i][0] = 0;
        for (int j = 0; j <= m; j++) pre[0][j] = 0;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                unsigned long long x;
                scanf("%llu", &x);
                pre[i][j] = x + pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1];
            }
        }
        unsigned long long answer = 0;
        for (int i = 0; i < q; i++) {
            int x1, y1, x2, y2;
            scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
            unsigned long long sum = pre[x2][y2] - pre[x1-1][y2]
                                   - pre[x2][y1-1] + pre[x1-1][y1-1];
            answer ^= sum;
        }
        printf("%llu\n", answer);
    }
    return 0;
}
```

3.45 P8649：k 倍区间

同类练习（在线做题）：P8649 [蓝桥杯 2017 省 B] k 倍区间。

题意。输入 n、k 和 n 项正整数，求和能被 k 整除的非空连续区间数。$n, k, a_i \le 10^5$。**解答**。区间和是 k 倍数当且仅当两端前缀余数相等。顺序扫描，每读到余数 r，先加以前的 cnt[r]，再增加计数。初始 cnt[0]=1 代表空前缀；次序反过来会把空区间算进去。$O(n+k)$ 时间、$O(k)$ 空间，答案

用 long long。**自测样例（按原题规则构造）**。

输入输出样例

样例输入

```
5 3
1 2 3 4 5
```

样例输出

```
7
```

完整参考实现 64_k_intervals.cpp

```cpp
#include <cstdio>
long long cnt[100005];
int main() {
    int n, k;
    scanf("%d%d", &n, &k);
    cnt[0] = 1;
    int remainder = 0;
    long long answer = 0;
    for (int i = 0; i < n; i++) {
        int x;
        scanf("%d", &x);
        remainder = (remainder + x) % k;
        answer += cnt[remainder];
        cnt[remainder]++;
    }
    printf("%lld\n", answer);
    return 0;
}
```

3.46 P1638：逛画展

同类练习（在线做题）：P1638 逛画展。

题意。n 幅画各有 1 到 m 的作者编号，求包含全部作者的最短连续区间；同长时输出左端点最小的一组，保证有解。$n \le 10^6, m \le 2000$。**解答**。频率记录窗口内作者出现次数，种类数达到 m 就尽量缩小左端。每次删除后若该作者计数变 0，窗口才失去一类。仅在长度严格变小时更新答案，扫描顺序会保留最小左端。$O(n+m)$。**自测样例（按原题规则构造）**。

输入输出样例

样例输入

```
7 3
1 2 1 3 2 1 3
```

样例输出

```
2 4
```

完整参考实现 65_gallery.cpp

```cpp
#include <cstdio>
int a[1000005], cnt[2005];
int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);
    int left = 1, kinds = 0, bestLeft = 1, bestRight = n;
    for (int right = 1; right <= n; right++) {
        if (cnt[a[right]]++ == 0) kinds++;
        while (kinds == m) {
            if (right - left < bestRight - bestLeft) {
                bestLeft = left;
                bestRight = right;
            }
            if (--cnt[a[left]] == 0) kinds--;
            left++;
        }
    }
    printf("%d %d\n", bestLeft, bestRight);
    return 0;
}
```

3.47 P3143：两个展示柜

同类练习（在线做题）：P3143 [USACO16OPEN] Diamond Collector S。

题意。n 颗钻石放进两个柜，每柜最大、最小尺寸差不超过 K；每颗只能使用一次，求总展示数最多。$n \le 50000$。**解答**。先排序。双指针求每个起点的最长合法段，再算后缀中单柜最多能取多少。第一个柜取 $[l, r)$，第二个柜从 r 以后取，贡献 len[l]+best[r]。有最优方案可按尺寸把两柜分开，扩大第一段、删去重叠仍不会降低总数；不能直接相加两个可能重叠的最大窗口。$O(n \log n)$。**自测样例（按原题规则构造）**。

输入输出样例

样例输入

```
7 2
1 2 3 8 9 10 20
```

样例输出

```
6
```

完整参考实现 66_diamonds.cpp

```cpp
#include <cstdio>
#include <algorithm>
int a[50005], len[50005], best[50006];
int main() {
    int n, k;
    scanf("%d%d", &n, &k);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    std::sort(a, a + n);
    int right = 0;
    for (int left = 0; left < n; left++) {
        while (right < n && a[right] - a[left] <= k) right++;
        len[left] = right - left;
    }
    for (int i = n - 1; i >= 0; i--)
        best[i] = std::max(len[i], best[i + 1]);
    int answer = 0;
    for (int i = 0; i < n; i++)
        answer = std::max(answer, len[i] + best[i + len[i]]);
    printf("%d\n", answer);
    return 0;
}
```

3.48 P10719：黑白格

同类练习（在线做题）：P10719 [GESP202406 五级] 黑白格。

题意。输入 n、m、k 及 01 网格，求至少包含 k 个黑格的最小子矩形面积；不存在输出 0。$n, m \le 100$。**解答**。枚举上下边界，累加每列黑格数，再用非负数组滑动窗口找和至少 k 的最短列区间。新增一行只需 $O(m)$，避免重新统计每个矩形。$O(n^2 m)$ 时间、$O(nm)$ 存储。缩小窗口时不要越过右端点；k 为正，零列也需要正确删除。**自测样例（按原题规则构造）**。

输入输出样例

样例输入

```
2 3 3
101
011
```

样例输出

```
4
```

完整参考实现 67_black_rectangle.cpp

```cpp
#include <cstdio>
#include <algorithm>
char grid[105][105];
int col[105];
int main() {
    int n, m, k;
    scanf("%d%d%d", &n, &m, &k);
    for (int i = 0; i < n; i++) scanf("%100s", grid[i]);
    int answer = n * m + 1;
    for (int top = 0; top < n; top++) {
        std::fill(col, col + m, 0);
        for (int bottom = top; bottom < n; bottom++) {
            for (int j = 0; j < m; j++) col[j] += grid[bottom][j] == '1';
            int left = 0, sum = 0;
            for (int right = 0; right < m; right++) {
                sum += col[right];
                while (sum >= k) {
                    int area = (bottom - top + 1) * (right - left + 1);
                    answer = std::min(answer, area);
                    sum -= col[left++];
                }
            }
        }
    }
    printf("%d\n", answer == n * m + 1 ? 0 : answer);
    return 0;
}
```

4 练习题

练习题型	做题链接
一维前缀和	P8218 求区间和
区间差分	P2367 语文成绩
二分查找	P2249 查找
二分答案与贪心判定	P2678 跳石头
双指针与重复计数	P1102 A-B 数对
区间选择贪心	P1803 凌乱的 yyy / 线段覆盖
DFS 与回溯	P1706 全排列问题
01 背包	P1048 采药
多重背包	P1776 宝物筛选

第三部分　第三题：字符串解析与综合模拟

1　字符串解析与综合模拟基础

1.1　字符数组与字符串操作

C 风格字符串是以结束符'\0' 结尾的字符数组。长度不包括结束符，容量必须包括它。strlen 求长度，strcmp 比较内容，strcpy 复制，头文件为 <cstring>。

```
char a[20] = "cat", b[20];
int len = (int)strlen(a); // 3.
strcpy(b, a);             // Copies the final '\0' too.
b[0] = 'b';               // b is now "bat".
int relation = strcmp(a, b); // Positive: "cat" > "bat".
```

strcmp 返回负数、0、正数，分别表示字典序小于、相等、大于；不能用数组名的 == 比较内容。复制前保证目标容量足够。strlen、strcpy 随字符串长度线性增长；strcmp 最坏需要检查完整公共前缀。循环处理字符时先保存长度，避免每轮重新调用 strlen。

1.2　整行输入与字符处理

fgets 最多读取容量减 1 个字符，保留读到的换行，并补结束符。若一行最多有 1000 个正文字符，下面的容量可以容纳正文、行尾和结束符。

```
char line[1003];
if (fgets(line, sizeof(line), stdin) != NULL)
    line[strcspn(line, "\r\n")] = '\0';
```

strcspn 返回第一个回车或换行的位置，无行尾时返回字符串长度；该位置置 0，只去掉行尾，不删除正文空格。若先 scanf 读数字，再 fgets 读下一行，应先消耗数字所在行的剩余字符：

```
int c;
while ((c = getchar()) != '\n' && c != EOF) {}
```

字符分类函数位于 <cctype>。isdigit 判断数字，isspace 判断空白，tolower 转为小写；参数先转为 unsigned char，避免有符号 char 的负值。

```
int length = (int)strlen(line), digits = 0;
for (int i = 0; i < length; i++) {
    unsigned char c = (unsigned char)line[i];
    if (isdigit(c)) digits++;
    line[i] = (char)tolower(c);
}
```

扫描长度 L 的文本为 $O(L)$。若规则只把普通空格当分隔符，就比较' '；isspace 还会接受制表符等空白。

1.3 扫描指针与状态机

扫描指针 p 表示下一个尚未处理的字符。连续数字应一次读取成一个数，而不是把每个字符单独当成数据。

```
// s is a null-terminated string; p initially points at a digit.
long long value = 0;
while (s[p] >= '0' && s[p] <= '9') {
    value = value * 10 + s[p] - '0';
    p++;
}
```

例如从 `12,7` 的开头读取，value 依次为 1、12，结束时 p 指向逗号。数值范围必须允许每次乘 10 再加数字。

状态机用少量状态说明同一字符在不同上下文中的含义。例如引号外的空格是分隔符，引号内的空格是正文；用 in_quote 记录当前是否在引号中。

```
// Input has matched quotes and no escaping.
int in_quote = 0, separators = 0;
for (int p = 0; s[p] != '\0'; p++) {
    if (s[p] == '"') in_quote = !in_quote;
    else if (s[p] == ' ' && !in_quote) separators++;
}
```

`a "b c" d` 的两个引号外空格被计数，引号内空格不计。每个字符触发一次状态转移，时间 $O(L)$，额外空间 $O(1)$。完整解析还需按规则决定空片段、转义和非法输入的处理方式。

1.4 状态存储与更新顺序

模拟用变量保存当前状态，按照操作规则更新。每个变量应对应明确的对象和时刻：编号作下标，多个属性放入结构体，查询读取该时刻已生效的状态。

1.4.1 同步更新

若一轮中的每个新值都依赖上一轮，必须从旧数组 a 读取，写入新数组 b；全部算完后再复制回 a。

```
// One round: new value = old value + old immediate neighbors.
for (int i = 0; i < n; i++) {
    b[i] = a[i];
    if (i > 0) b[i] += a[i-1];
    if (i+1 < n) b[i] += a[i+1];
}
for (int i = 0; i < n; i++) a[i] = b[i];
```

$[1, 2, 3]$ 一轮后为 $[3, 6, 5]$。若直接改 a，计算中间位置时会读到已更新的 3，结果变为 8。每轮 $O(n)$ 时间，两个数组 $O(n)$ 空间。

1.4.2 顺序更新

若操作依次生效，后一条必须看到前一条的结果。事件模拟还要确定同一时刻的先后，例如先加入该时刻到达的对象，再从全部候选中选择。

```
// Events are already sorted by arrival time.
while (next < n && events[next].arrival <= now) {
    add_candidate(events[next]);
    next++;
}
if (has_candidate()) execute_one();
```

小于等于把恰好在 now 到达的对象也纳入本次选择。每个事件只加入一次，加入阶段总计处理 n 项；选择和执行成本由相应数据结构决定。

1.5 栈与递归解析

1.5.1 栈保存未完成状态

处理嵌套结构时，最近打开的结构应最先关闭，用栈保存尚未配对的左括号。

```
// s contains only '(' and ')'; stack capacity >= strlen(s).
int top = 0, valid = 1;
for (int p = 0; s[p] != '\0'; p++) {
    if (s[p] == '(') st[top++] = p;
    else if (top == 0) { valid = 0; break; }
    else top--;
}
if (top != 0) valid = 0;
```

右括号遇到空栈表示没有对应左括号，扫描结束仍有元素表示左括号未关闭。栈中存位置还可以记录配对关系。时间 $O(L)$，最坏空间 $O(L)$。

1.5.2 递归函数对应语法层级

递归解析把语法中的一个结构交给一个函数。以非负整数、加号和括号为例：

$$\text{expr} = \text{factor}\ \{+\ \text{factor}\},$$
$$\text{factor} = \text{number} \mid (\ \text{expr}\ ).$$

expr 处理连续加法，factor 处理数字或括号。共享指针 p 始终指向下一个未读字符；本片段假定语法合法、没有空白，数值运算不溢出。

```
char s[1005]; int p = 0;
long long parse_expr();
long long parse_factor() {
    if (s[p] == '(') {
        p++;
        long long value = parse_expr();
        p++; // Consume the matching ')'.
        return value;
    }
    long long value = 0;
    while (s[p] >= '0' && s[p] <= '9')
        value = value*10 + s[p++] - '0';
    return value;
}
long long parse_expr() {
    long long value = parse_factor();
    while (s[p] == '+') {
        p++;
        value += parse_factor();
    }
    return value;
}
```

解析 `1+(2+3)` 时，外层读到左括号便递归求出 5，再与 1 相加；内层在右括号处停止，由调用它的 factor 消费右括号。每个字符消费一次，时间 $O(L)$，调用栈空间 $O(H)$，H 为嵌套深度。若题目允许非法输入，需要检查预期的字符，并在顶层确认整行读完。

1.6 树形结构与依赖处理

1.6.1 父子关系与子树

树形结构用父编号或孩子列表表达。普通数组可以用 first[u] 保存第一个孩子，next[v] 保存同父节点下的下一个孩子。

```
// Nodes 0..n-1; parent[root] = -1.
for (int u = 0; u < n; u++) first[u] = next[u] = -1;
for (int v = 0; v < n; v++) {
    int u = parent[v];
    if (u == -1) continue;
    next[v] = first[u];
    first[u] = v;
}
```

孩子按逆插入顺序连接；若题目要求输入顺序，需维护尾指针或按反向编号插入。递归遍历孩子后再处理父亲，可以汇总子树信息：

```
int subtree_size(int u) {
    int total = 1;
    for (int v = first[u]; v != -1; v = next[v])
        total += subtree_size(v);
    return total;
}
```

返回值包含 u 本身。合法树上每个节点和父子边访问一次，时间 $O(n)$，存储 $O(n)$，调用栈深度为树高 H；链状树可能达到 n 层。

1.6.2 依赖顺序与环检测

若 u 的结果依赖 v，就应先完成 v。依赖图不一定是树：多个对象可以依赖同一个对象，也可能出现环。用三种颜色记录进度：0 未开始，1 正在当前递归链中，2 已经完成。

```
// dependency[u] lists objects needed by u; color is initially zero.
bool solve(int u) {
    if (color[u] == 1) return false;
    if (color[u] == 2) return true;
    color[u] = 1;
    for (int i = 0; i < (int)dependency[u].size(); i++)
        if (!solve(dependency[u][i])) return false;
    color[u] = 2;
    order[count++] = u;
    return true;
}
```

再次遇到颜色 1，说明当前链回到尚未结束的对象，存在环；遇到颜色 2 则复用已经完成的结果，不是环。order 中依赖对象先于使用它的对象，可以按此顺序计算。对所有未访问节点调用 solve 即可检查全图，n 个对象、m 条依赖的时间与存储为 $O(n+m)$；实际结果计算和输出若更大，应另计成本。

2 例题速看

2.1 带引号的单行分词（教学例题）

题目。一行文本由普通字符、空格和英文双引号组成。引号外的连续空格分隔单词；引号内的空格属于单词；双引号本身不输出。相邻的普通片段和引号片段属于同一个单词。空引号表示一个空单词。本题不支持转义字符。若引号不匹配，输出 ERROR。行长不超过 10^5。

输入。一整行文本，可为空。**输出**。合法时先输出单词数，随后每个单词单独一行，用方括号包住，便于看见空单词和空格。

输入输出样例

样例输入

```
add "hello world" "" x
```

样例输出

```
4
[add]
[hello world]
[]
[x]
```

状态设计。quoted 表示当前是否在引号内；word[0..len-1] 保存当前单词；active 表示当前单词已经开始。active 不能由 len 是否为 0 代替，否则空引号会丢失。遇到引号切换 quoted；遇到引号外空格时结束当前单词；其余字符追加到 word 并增加 len。扫描结束后检查引号是否闭合，再处理最后一个单词。

正确性。每个字符恰好进入一种分支。引号仅改变状态，分隔空格仅结束一个已开始的单词，因此保留的字符、单词边界和空单词均符合规则。时间 $O(L)$，输出存储 $O(L)$。

完整参考实现 04_tokens.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <string>
#include <vector>
using namespace std;
const int N = 100005;
char s[N], word[N];
int main() {
    if (fgets(s, N, stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
    vector<string> words;
    int len = 0, quoted = 0, active = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '"') { quoted = !quoted; active = 1; }
        else if (c == ' ' && !quoted) {
            if (active) {
                word[len] = '\0'; words.push_back(word);
                len = 0; active = 0;
            }
        } else { word[len++] = c; active = 1; }
    }
    if (quoted) { puts("ERROR"); return 0; }
    if (active) { word[len] = '\0'; words.push_back(word); }
    printf("%d\n", (int)words.size());
    for (int i = 0; i < (int)words.size(); i++)
        printf("[%s]\n", words[i].c_str());
    return 0;
}
```

2.2 例题路径规范化

真题练习：路径解析：第 7 次认证，处理绝对路径、相对路径以及点号路径段（CSP201604C）。

题目。给定固定的当前目录 cwd 和 q 条路径。以斜杠开头的是绝对路径，否则相对于 cwd。连续斜杠视为一个；点号目录表示当前目录；两个点号表示父目录，在根目录继续上移仍为根。普通目录名只含字母、数字、下划线。路径不含空格且非空。

输出规范绝对路径：以一个斜杠开头，不含点号、两个点号或重复斜杠；除根目录外不以斜杠结尾。**每条查询独立使用原来的 cwd，不改变当前目录**。路径长度不超过 10^4，全部输入路径总长度不超过 2×10^5，$1 \le q \le 10^4$。第一行 cwd，第二行 q，随后 q 行各一条路径。cwd 保证为绝对路径，也允许特殊片段。

输入输出样例

样例输入

```
/a/b
5
../c
/x//./y/../
../../../
.
tmp/../../z
```

样例输出

```
/a/c
/x
/
/a/b
/a/z
```

解答。用 vector 保存从根向下的目录名。普通片段入栈，两个点号在非空时弹栈，空片段和点号忽略。绝对路径从空栈开始，相对路径从 cwd 的副本开始。

最后一条查询从 $[a, b]$ 出发，读 tmp 得到 $[a, b, tmp]$，两次父目录操作后为 $[a]$，加入 z 后得到 $[a, z]$。不变量是：栈中的目录始终表示已经处理的路径前缀的规范形式。

设 cwd 长度为 B，第 i 个查询长度为 L_i。复制基目录的总成本最坏为 $O(qB)$，加上扫描与输出为 $O(qB + \sum L_i)$。不能漏算复制和输出中重复的 cwd 长度。

完整参考实现 19_path.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <string>
#include <vector>
using namespace std;
const int N = 10005;
char cwd[N], path[N];
void append_parts(const char s[], vector<string> &parts) {
    char item[N];
    int i = 0;
    while (s[i] != '\0') {
        while (s[i] == '/') i++;
        int len = 0;
        while (s[i] != '\0' && s[i] != '/') item[len++] = s[i++];
        item[len] = '\0';
        if (len == 0 || strcmp(item, ".") == 0) continue;
        if (strcmp(item, "..") == 0) {
            if (!parts.empty()) parts.pop_back();
        } else parts.push_back(item);
    }
}
int main() {
    int q;
    scanf("%10004s%d", cwd, &q);
    vector<string> base;
    append_parts(cwd, base);
    while (q--) {
        scanf("%10004s", path);
        vector<string> result;
        if (path[0] != '/') result = base;
        append_parts(path, result);
        if (result.empty()) printf("/");
        for (int i = 0; i < (int)result.size(); i++)
            printf("/%s", result[i].c_str());
        printf("\n");
    }
    return 0;
}
```

2.3 例题授权和撤销的状态管理

真题练习：角色授权：第 26 次认证；下面教学例题先练习较小的授权模型，原题还需处理角色关联与权限匹配（CSP202206C）。

题目。执行 q 条命令：`GRANT user resource` 授予权限；`REVOKE user resource` 撤销；`CHECK user resource` 查询并输出 YES 或 NO。重复授予不叠加；撤销不存在的权限不报错；未出现用户默认无权限。操作依次执行，只有 CHECK 输出。$1 \le q \le 2 \times 10^5$，名称为 1 到 20 个字母或数字。本题不含角色继承、通配符或优先级。

输入输出样例

样例输入

```
8
CHECK alice file1
GRANT alice file1
GRANT alice file1
CHECK alice file1
REVOKE alice file1
CHECK alice file1
REVOKE bob file2
CHECK bob file2
```

样例输出

```
NO
YES
NO
NO
```

解答。外层 map 按用户名定位，内层 set 保存资源。set 处理重复授权。查询和撤销用 find，避免无意创建空记录。所有操作依次执行，因此 CHECK 反映此前的状态。

设用户数 U，每位用户最多持有 R 项权限，单次比较次数为 $O(\log U + \log R)$，还需考虑字符串比较长度。总存储不超过授予过的记录规模。权限是用户与资源的二元关系，不能只判断用户是否出现过；撤销 file1 不能影响 file2。

完整参考实现 20_permissions.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <map>
#include <set>
#include <string>
using namespace std;
map<string, set<string> > rights;
int main() {
    int q;
    scanf("%d", &q);
    while (q--) {
        char op[10], user[21], resource[21];
        scanf("%9s%20s%20s", op, user, resource);
        if (strcmp(op, "GRANT") == 0) rights[user].insert(resource);
        else {
            map<string, set<string> >::iterator it = rights.find(user);
            if (strcmp(op, "REVOKE") == 0) {
                if (it != rights.end()) it->second.erase(resource);
            } else {
                int allowed = it != rights.end() && it->second.count(resource);
                puts(allowed ? "YES" : "NO");
            }
        }
    }
    return 0;
}
```

2.4 例题带括号的整数表达式

题目。输入一行表达式，支持非负整数字面量、加、减、乘、括号和一元正负号。乘法优先于加减，同级二元运算从左到右；一元符号优先于乘法；只忽略普通空格。合法时输出结果，语法错误输出 ERROR。不支持除法、变量和隐式乘法，例如 `2(3+4)` 非法。

行长不超过 1000，括号与连续一元符号嵌套不超过 100 层，字面量不超过 10^6；保证求值过程中涉及的数值运算的中间结果绝对值不超过 10^{12}，本题只讨论语法错误，不要求处理溢出。

输入输出样例 1

样例输入

```
2 * (3 + 4) - -5
```

样例输出

```
19
```

输入输出样例 2

样例输入

```
1 + (2 * 3
```

样例输出

```
ERROR
```

按语法拆函数。

语法规则

```
expression = term { ('+' | '-') term }
term       = factor { '*' factor }
factor     = number | '(' expression ')' | '+' factor | '-' factor
```

expression 负责加减，term 负责乘法，factor 负责括号、数字与一元符号。共享指针 p 指向下一个未处理字符，每个函数返回其负责部分的值。term 先完整处理乘法，从而保证优先级；循环保证减法左结合，例如 $8-3-2=3$。

解析前面一个合法表达式不代表整行合法。例如 `1 2`，解析 1 后还剩 2，必须输出 ERROR。最后检查是否读到行尾。时间 $O(L)$，递归空间与嵌套深度有关。

完整参考实现 21_expression.cpp

```cpp
#include <cstdio>
#include <cstring>
char s[1005];
int p = 0, valid = 1;
void skip_space() { while (s[p] == ' ') p++; }
int take(char c) {
    skip_space();
    if (s[p] == c) { p++; return 1; }
    return 0;
}
long long expression();
long long factor() {
    if (take('+')) return factor();
    if (take('-')) return -factor();
    if (take('(')) {
        long long x = expression();
        if (!take(')')) valid = 0;
        return x;
    }
    skip_space();
    if (s[p] < '0' || s[p] > '9') { valid = 0; return 0; }
    long long x = 0;
    while (s[p] >= '0' && s[p] <= '9') x = x * 10 + s[p++] - '0';
    return x;
}
long long term() {
    long long x = factor();
    while (valid && take('*')) x *= factor();
    return x;
}
long long expression() {
    long long x = term();
    while (valid) {
        if (take('+')) x += term();
        else if (take('-')) x -= term();
        else break;
    }
    return x;
}
int main() {
    if (fgets(s, sizeof(s), stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
    long long answer = expression();
    skip_space();
    if (!valid || s[p] != '\0') puts("ERROR");
    else printf("%lld\n", answer);
    return 0;
}
```

2.5 例题树形文档的后代选择查询

真题练习：元素选择器：第 14 次认证；下面先用显式父编号练习祖先链，原题还需解析缩进与选择器（CSP201809C）。

题目。给出按先序排列的一片有根森林，共 n 个节点。每行记录深度、标签和标识。根深度 0，孩子比父亲深 1；输入结构合法，每次深度最多增加 1。标识为减号表示无标识，其余标识互不相同；标签和标识只含字母数字，长度 1 到 20。

查询由空格分隔的选择符组成。普通选择符匹配标签，以井号开头的选择符匹配标识。`div p` 表

示匹配 p 节点，且它的某个**严格祖先**是 div；`div div p` 需要两个不同的 div 祖先按顺序出现，允许隔着其他节点。节点不能把自己当祖先。

$1 \le n \le 500$，$1 \le q \le 50$，最大深度 100，每条查询 1 到 10 个选择符，查询行总长度不超过 1000。第一行 n、q，随后 n 行节点，最后 q 行查询。每次输出匹配数和按输入顺序排列的节点行号，编号从 1 开始。

输入输出样例

样例输入

```
7 5
0 html -
1 body -
2 div main
3 p one
2 div side
3 p two
2 p three
p
div p
#main p
div div p
#side
```

样例输出

```
3 4 6 7
2 4 6
1 4
0
1 5
```

建树。用祖先栈。读取深度 d 的节点时，把栈截断到长度 d；栈顶就是父亲，空栈则为根，然后压入自己。读取同深度兄弟时，旧兄弟会被弹出。

匹配。从查询末尾开始，先检查候选节点本身，再沿父链逆序匹配剩余选择符。找到一个就继续寻找更高祖先。选择最近的匹配祖先不会损失可行解，因为它给剩余前缀保留最多的上方祖先。找齐全部选择符则接受。

候选检查最多走树高 H 条父边并处理 K 个选择符，整体为 $O(qn(H+K))$，本题规模下足够。节点 7 虽是 p，但没有 div 祖先；若程序把它算入 `div p`，通常是把“之前出现的节点”误认为“当前祖先”。

完整参考实现 22_selector.cpp

```cpp
#include <cstdio>
#include <cstring>
struct Node { char tag[21], id[21]; int parent; };
Node a[505];
int ancestors[505], answer[505];
char line[1005], tokens[10][22];
int matches(int u, const char token[]) {
    if (token[0] == '#')
        return strcmp(a[u].id, "-") != 0 && strcmp(a[u].id, token + 1) == 0;
    return strcmp(a[u].tag, token) == 0;
}
int main() {
    int n, q, top = 0;
    scanf("%d%d", &n, &q);
    for (int i = 0; i < n; i++) {
        int depth;
        scanf("%d%20s%20s", &depth, a[i].tag, a[i].id);
        top = depth;
        a[i].parent = top == 0 ? -1 : ancestors[top - 1];
        ancestors[top++] = i;
    }
    int ch;
    while ((ch = getchar()) != '\n' && ch != EOF) {}
    while (q--) {
        if (fgets(line, sizeof(line), stdin) == NULL) break;
        int count = 0;
        char *s = strtok(line, " \t\r\n");
        while (s != NULL) {
            strcpy(tokens[count++], s);
            s = strtok(NULL, " \t\r\n");
        }
        int total = 0;
        for (int u = 0; u < n; u++) {
            int k = count - 1;
            if (k < 0 || !matches(u, tokens[k])) continue;
            k--;
            int v = a[u].parent;
            while (v != -1 && k >= 0) {
                if (matches(v, tokens[k])) k--;
                v = a[v].parent;
            }
            if (k < 0) answer[total++] = u + 1;
        }
        printf("%d", total);
        for (int i = 0; i < total; i++) printf(" %d", answer[i]);
        printf("\n");
    }
    return 0;
}
```

2.6 例题模板展开与循环依赖

真题练习：模板生成系统：第 5 次认证；下面教学题额外加入递归依赖与环检测，原题按其自身替换规则实现（CSP201509C）。

题目。有 n 个命名模板，每个由若干以空白分隔的词组成。以 at 符号开头的词引用另一个模板，其余词直接输出。定义全部读完后，展开指定起始模板，以单个空格连接所有普通词。从起始模板可达的引用若出现未定义名称或环，输出 ERROR；不可达模板的错误不影响本次输出。

模板名互不重复，只含字母数字，长度不超过 20；引用符号后必须有名称。允许空模板。$1 \le$

$n \le 100$, 每个定义最多 100 个词, 每词长度最多 20。保证无环情况下每个模板完整展开不超过 2000 个词。第一行 n，随后每个定义包含名称、词数 k 及 k 个词，最后给出起始名称。

输入输出样例 1

样例输入

```
3
title 1 world
line 3 hello @title !
all 2 @line @line
all
```

样例输出

```
hello world ! hello world !
```

输入输出样例 2

样例输入

```
2
a 1 @b
b 1 @a
a
```

样例输出

```
ERROR
```

三色状态。0 表示未算，1 表示正在递归，2 表示已经完成并缓存。访问状态 1 说明当前依赖链回到了自己；访问状态 2 直接复用。第二次引用同一模板不必然成环，关键看第一次展开是否结束。

解答。先读入全部定义以允许前向引用。按词序展开，普通词加入结果，引用词接上被引用模板的结果。记忆化避免重算，但每次拼接仍复制相应词。时间取决于查找及所有实际复制的词数；本题每个模板最多缓存 2000 词，总缓存约不超过 100×2000 词，不能忽略输出规模而声称总是 O(n)。

完整参考实现 23_templates.cpp

```cpp
#include <cstdio>
#include <map>
#include <string>
#include <vector>
using namespace std;
map<string, int> number;
vector<string> definition[105], memo[105];
int state[105];
int expand(int u) {
    if (state[u] == 1) return 0;
    if (state[u] == 2) return 1;
    state[u] = 1;
    for (int i = 0; i < (int)definition[u].size(); i++) {
        string token = definition[u][i];
        if (token[0] != '@') memo[u].push_back(token);
        else {
            map<string, int>::iterator it = number.find(token.substr(1));
            if (it == number.end() || !expand(it->second)) return 0;
            int v = it->second;
            for (int j = 0; j < (int)memo[v].size(); j++)
                memo[u].push_back(memo[v][j]);
        }
    }
    state[u] = 2;
    return 1;
}
int main() {
    int n;
    scanf("%d", &n);
    char name[21], token[21];
    for (int i = 0; i < n; i++) {
        int k;
        scanf("%20s%d", name, &k); number[name] = i;
        for (int j = 0; j < k; j++) {
            scanf("%20s", token); definition[i].push_back(token);
        }
    }
    scanf("%20s", name);
    map<string, int>::iterator it = number.find(name);
    if (it == number.end() || !expand(it->second)) puts("ERROR");
    else {
        int u = it->second;
        for (int i = 0; i < (int)memo[u].size(); i++)
            printf("%s%s", i == 0 ? "" : " ", memo[u][i].c_str());
        printf("\n");
    }
    return 0;
}
```

2.7 例题差值数对

给 n 个整数和非负整数 c，求满足 $i < j$ 且 $|a_i - a_j| = c$ 的位置对数量。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$，$0 \le c \le 2 \times 10^9$。第一行 n、c，第二行数组，输出一个整数。

输入输出样例

样例输入

```
5 2
1 3 1 3 3
```

样例输出

```
6
```

解答。先统计每个值的频次。c>0 时，只从较小值 x 寻找 x+c，不会重复统计同一无序数对；贡献是两组频次乘积。c=0 时，频次 k 的组中任选两个位置，贡献 $k(k-1)/2$。答案最多为 $n(n-1)/2$，需要 long long。时间 $O(n \log n)$，空间 $O(n)$。

完整参考实现 `24_pairs.cpp`

```cpp
#include <cstdio>
#include <map>
using namespace std;
int main() {
    int n; long long c;
    scanf("%d%lld", &n, &c);
    map<long long, long long> count;
    for (int i = 0; i < n; i++) {
        long long x; scanf("%lld", &x); count[x]++;
    }
    long long answer = 0;
    map<long long, long long>::iterator it;
    for (it = count.begin(); it != count.end(); ++it) {
        long long x = it->first, k = it->second;
        if (c == 0) answer += k * (k - 1) / 2;
        else {
            map<long long, long long>::iterator p = count.find(x + c);
            if (p != count.end()) answer += k * p->second;
        }
    }
    printf("%lld\n", answer);
    return 0;
}
```

检查用例。数组 $[7,7,7]$、c=0 应输出 3；单元素时应输出 0。若输出 9，说明把相同位置配对或对称重复都算了进去。

2.8 例题矩形增量的最大值

初始有 $n \times m$ 全 0 矩阵，执行 q 次操作，每次给闭矩形左上 (x_1, y_1) 到右下 (x_2, y_2) 加 v。最终输出最大元素值，以及等于最大值的格子数。$1 \le n, m \le 1000$，$0 \le q \le 2 \times 10^5$，$|v| \le 10^6$，坐标合法。第一行 n、m、q，随后每行五个数。

输入输出样例

样例输入

```
3 3 2
1 1 2 2 2
2 2 3 3 3
```

样例输出

```
5 1
```

解答。把每次矩形更新记录在二维差分四角，最后恢复每格真实值，同时维护最大值和次数。只统计 1 到 n 行、1 到 m 列，不能把辅助边界算进去。允许负更新，最大值要从负无穷开始，不能默认 0。时间 $O(nm+q)$，空间 $O(nm)$。

完整参考实现 25_rectangle.cpp

```cpp
#include <cstdio>
#include <climits>
const int N = 1005;
long long d[N][N];
int main() {
    int n, m, q;
    scanf("%d%d%d", &n, &m, &q);
    while (q--) {
        int x1, y1, x2, y2; long long v;
        scanf("%d%d%d%d%lld", &x1, &y1, &x2, &y2, &v);
        d[x1][y1] += v; d[x2 + 1][y1] -= v;
        d[x1][y2 + 1] -= v; d[x2 + 1][y2 + 1] += v;
    }
    long long best = LLONG_MIN, count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            d[i][j] += d[i - 1][j] + d[i][j - 1] - d[i - 1][j - 1];
            if (d[i][j] > best) { best = d[i][j]; count = 1; }
            else if (d[i][j] == best) count++;
        }
    }
    printf("%lld %lld\n", best, count);
    return 0;
}
```

检查用例。q=0 时输出 0 n*m，第二项应计算成实际格子数；若整个矩阵一次减 5，则最大值为-5 而不是 0。

2.9 例题非抢占任务调度

有一台处理器和 n 个任务，每个任务给定到达时刻、优先级和执行时间，输入顺序就是任务编号。处理器空闲时，在所有已经到达但尚未执行的任务中选择优先级最高者；并列时选到达更早者，再并列选编号更小者。一旦开始执行就**不能被抢占**。

一个任务完成时，必须先把恰好在该时刻到达的任务加入候选，再选择下一项。若没有已到达任务，时钟直接跳到下一次到达。输入不保证按到达时刻有序。$1 \le n \le 2 \times 10^5$，到达时刻和优先级在 0 到 10^9 之间，执行时间在 1 到 10^9 之间。

第一行 n，随后每行依次给到达时刻、优先级、执行时间。按输入编号输出每个任务的开始和结束时刻，最后一行输出最大等待时间，等待时间为开始减到达。

输入输出样例

样例输入

```
4
0 1 5
1 1 2
2 3 1
5 2 2
```

样例输出

```
0 5
8 10
5 6
6 8
7
```

推演。任务 1 在 0 到 5 执行，中途到达的高优先级任务不能打断它。时刻 5，先加入任务 4；待选任务 2、3、4 中任务 3 优先级最高，所以执行顺序为 1、3、4、2。任务 2 等到 8 开始，等待 7。

状态划分。排序数组保存还未到达的任务，指针 i 指向下一项；优先队列保存已经到达的任务；now 保存处理器可再次分配工作的时刻。每轮先补全所有到达时间不大于 now 的任务，再选堆顶。

正确性。选择前，堆恰好包含全部可选任务；比较器严格实现三级优先顺序。执行结束后更新 now，不处理中途抢占，符合规则。每个任务入堆出堆各一次，排序与堆操作总时间 $O(n \log n)$，空间 $O(n)$。结束时刻可能达到 10^{14} 量级，应使用 long long。

完整参考实现 26_scheduler.cpp

```cpp
#include <cstdio>
#include <algorithm>
#include <queue>
#include <vector>
using namespace std;
const int N = 200005;
struct Task { long long arrival, priority, duration; int id; };
Task tasks[N];
long long start_time[N], finish_time[N];
bool by_arrival(Task a, Task b) {
    if (a.arrival != b.arrival) return a.arrival < b.arrival;
    return a.id < b.id;
}
bool lower_priority(Task a, Task b) {
    if (a.priority != b.priority) return a.priority < b.priority;
    if (a.arrival != b.arrival) return a.arrival > b.arrival;
    return a.id > b.id;
}
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%lld%lld%lld", &tasks[i].arrival,
              &tasks[i].priority, &tasks[i].duration);
        tasks[i].id = i;
    }
    sort(tasks, tasks + n, by_arrival);
    typedef bool (*Compare)(Task, Task);
    priority_queue<Task, vector<Task>, Compare> ready(lower_priority);
    long long now = 0, max_wait = 0;
    int i = 0;
    while (i < n || !ready.empty()) {
        if (ready.empty()) now = max(now, tasks[i].arrival);
        while (i < n && tasks[i].arrival <= now) ready.push(tasks[i++]);
        Task t = ready.top(); ready.pop();
        start_time[t.id] = now; finish_time[t.id] = now + t.duration;
        max_wait = max(max_wait, now - t.arrival);
        now += t.duration;
    }
    for (int j = 0; j < n; j++)
        printf("%lld %lld\n", start_time[j], finish_time[j]);
    printf("%lld\n", max_wait);
    return 0;
}
```

3 练习题

练习题型	做题链接
扫描与栈维护	P1106 删数问题
字符网格与传送规则	P1825 Corn Maze S
棋盘状态编码与更新	P1225 黑白棋游戏